TIS Project Reference Booklet

Documentation version: 3.0

Generated: 2026-06-26 13:06 Arab Standard Time

Branch: dev

Git commit SHA: 7d8c11d

Source of truth: Markdown files under docs/ are authoritative. This PDF is a generated snapshot and must not be edited manually.

Source Documents Included

- docs/README.md
- docs/AI_PROJECT_CONTEXT.md
- docs/DOCUMENTATION_UPDATE_POLICY.md
- docs/TIS_MASTER_CONTEXT.md
- docs/PROJECT_STATE.md
- docs/CHANGE_HISTORY.md
- docs/adr/README.md
- docs/engineering/README.md
- docs/engineering/TIS_MODULE_MAP.md
- docs/engineering/REPOSITORY_ARCHITECTURE.md
- docs/engineering/USER_AND_SYSTEM_FLOWS.md
- docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- docs/engineering/DEVELOPMENT_STANDARDS.md
- docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
- docs/engineering/PRODUCT_ROADMAP.md
- docs/adr/0001-separate-nextjs-landing-website.md
- docs/adr/0002-separate-saas-identity-and-operational-users.md
- docs/adr/0003-paddle-payment-architecture.md
- docs/adr/0004-webhook-only-payment-confirmation.md
- docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
- docs/adr/0006-documentation-as-source-knowledge-management-system.md
- docs/adr/0007-landing-page-visual-system-strategy.md
- docs/history/academic-calendar/README.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md
- docs/history/engineering-handbook/README.md
- docs/history/landing-page/README.md
- docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
- docs/history/platform-knowledge/README.md
- docs/history/provisioning/2026-06-26-kms-foundation.md
- docs/history/provisioning/README.md
- docs/history/README.md
- docs/history/saas-onboarding/README.md
- docs/history/subscriptions/README.md
- docs/history/workforce-planning/README.md
- docs/marketing/landing_page_source_of_truth.md
- docs/marketing/tis_landing_page_master_content.md
- docs/location-data-roadmap.md

docs/README.md

TIS Documentation

This folder is the source of truth for the TIS Knowledge Management System (KMS).

Markdown files are authoritative. The PDF booklet is a generated snapshot and must never be edited manually.

First Read For AI Coding Conversations

For any new Codex or ChatGPT coding conversation, load this file first:

1. `docs/AI_PROJECT_CONTEXT.md`

Then load:

1. `docs/TIS_MASTER_CONTEXT.md`
1. `docs/PROJECT_STATE.md`
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`
1. relevant ADRs under `docs/adr/`
1. relevant module history under `docs/history/`
1. engineering handbook docs under `docs/engineering/`

If the task touches the public website, also read:

1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`
1. `docs/adr/0001-separate-nextjs-landing-website.md`
1. `docs/adr/0007-landing-page-visual-system-strategy.md`

Core Documents

1. `docs/AI_PROJECT_CONTEXT.md`: compact onboarding file for future Codex and ChatGPT conversations.
1. `docs/TIS_MASTER_CONTEXT.md`: long-term product, architecture, SaaS, workflow, roadmap, and critical-rules source of truth.
1. `docs/PROJECT_STATE.md`: living status file for branch strategy, priority, milestones, known issues, and next work.
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`: non-negotiable KMS and Knowledge Impact Assessment policy.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.

Engineering Handbook

1. `docs/engineering/README.md`: engineering onboarding order and handbook index.
1. `docs/engineering/TIS_MODULE_MAP.md`: complete module map with purpose, files, maturity, docs, risks, and guardrails.
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md`: repository structure and ownership boundaries.

1. `docs/engineering/USER_AND_SYSTEM_FLOWS.md`: public customer, SaaS identity, payment, provisioning, operational login, platform owner, KMS, and developer onboarding flows.
1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`: conceptual data model and tenant/identity isolation rules.
1. `docs/engineering/DEVELOPMENT_STANDARDS.md`: non-negotiable engineering rules.
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`: UI/UX principles by product surface.
1. `docs/engineering/PRODUCT_ROADMAP.md`: completed, current, next, and future roadmap.

Decision And History Documents

1. `docs/adr/`: Architecture Decision Records for major accepted decisions.
1. `docs/history/`: module-based history preserving deeper before/after context.

Current module history areas:

1. `docs/history/subscriptions/`
1. `docs/history/landing-page/`
1. `docs/history/academic-calendar/`
1. `docs/history/workforce-planning/`
1. `docs/history/saas-onboarding/`
1. `docs/history/provisioning/`
1. `docs/history/platform-knowledge/`
1. `docs/history/engineering-handbook/`

Supporting Documents

1. `docs/location-data-roadmap.md`: location data roadmap and related implementation notes.
1. `docs/marketing/landing_page_source_of_truth.md`: boundary between the public Next.js landing website and the FastAPI application portal.
1. `docs/marketing/tis_landing_page_master_content.md`: approved marketing foundation and landing page content direction.

Generated Snapshot

Generated files:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`
1. `static/docs/docs_manifest.json`

The PDF is generated from approved Markdown docs. The manifest records the generated timestamp, documentation version, branch, commit SHA, included sources, and source hashes.

PDF Generation

Regenerate the PDF booklet with:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

The generator uses existing `reportlab` only. It must not require LaTeX, Playwright, Chromium, network calls, or system fonts.

Knowledge Impact Assessment

Every future implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included source docs change, regenerate the PDF.

Phase Boundary

Phase 2A and Phase 2B establish KMS governance, ADRs, module history, AI context, and PDF/manifest generation.

KMS v3.0 Phase 3B expands the engineering handbook with database architecture, development standards, UI/UX philosophy, roadmap, and stronger onboarding guidance. The Platform Owner Knowledge Center is implemented as a read-only owner utility. Do not add a regenerate button or app-side Markdown rewriting unless explicitly approved.

docs/AI_PROJECT_CONTEXT.md

TIS AI Project Context

This is the first file future Codex or ChatGPT coding conversations should load. It is a compact project onboarding reference; detailed source of truth remains in the other Markdown docs.

What TIS Is

TIS is Teacher Information System, a developing SaaS academic operations platform for schools and school groups. It connects teacher information, staffing and workload planning, academic calendars, observations, branch context, SaaS onboarding, billing, provisioning, and future AI-assisted academic decision support.

Public URLs:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`

Important routes:

1. Operational login: `/login`
1. SaaS signup: `/saas/signup`
1. SaaS login: `/saas/login`
1. SaaS account: `/saas/account`
1. Platform console: `/platform`

Current Architecture

The operational app is a FastAPI application at the repository root.

Key files and folders:

1. `main.py`: primary FastAPI app and many route handlers.
1. `auth.py`: authentication, roles, platform identity helpers, permissions, sessions.
1. `authorization.py`: protected route rules and access-denied handling.
1. `permission_registry.py`: permission keys, groups, defaults, and developer-assignable permissions.
1. `ui_shell.py`: shared app shell/navigation/page metadata.
1. `models.py`, `database.py`, `db_migrations.py`: data model, DB setup, local schema repair/migration logic.
1. `routers/`: modular operational routes.
1. `saas/`: SaaS account, onboarding, payment, billing, and provisioning services/routes.
1. `templates/`: Jinja templates.
1. `static/`: static assets and generated documentation output.
1. `tests/`: pytest coverage.

The public marketing website is separate:

1. `tis-landing-website/`

1. Next.js / Node runtime
1. Source of truth for public landing implementation

Legacy FastAPI landing files are not the source of truth:

1. `templates/landing.html`
1. `static/landing/landing.css`

Engineering Handbook

For deeper onboarding, read:

1. `docs/engineering/README.md`
1. `docs/engineering/TIS_MODULE_MAP.md`
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md`
1. `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`
1. `docs/engineering/DEVELOPMENT_STANDARDS.md`
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`
1. `docs/engineering/PRODUCT_ROADMAP.md`

These files explain module ownership, repository boundaries, end-to-end flows, and what must not be changed casually.

Domains And Routing

The public website lives at `https://tisplatform.com`. The app portal lives at `https://app.tisplatform.com`.

SaaS account routes are under `/saas`. Platform-owner SaaS administration routes are under `/saas-admin`. Operational tenant workflows use routes such as `/dashboard`, `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, and `/observations`.

Completed M1-M5 Milestones

M1: SaaS identity foundation and separation between platform, tenant, and SaaS account identities.

M2: SaaS onboarding flow for organization, contacts, branches, academic setup, and review.

M3: Billing and plan foundation with plan catalog, checkout, billing status, and payment service boundaries.

M4: Tenant provisioning foundation with pending organizations, provisioning jobs, retry/run actions, and platform owner oversight.

M5: Platform access and owner controls, including platform owner/developer identities, permissions, and platform console behavior.

Current Priority

Current priority is the TIS Knowledge Management System:

1. Markdown is source of truth.
1. PDF is generated snapshot.

1. Change history preserves chronological change context.
1. ADRs preserve major decisions.
1. Module history preserves deeper area-specific evolution.
1. Platform Owner Knowledge Center is implemented as a read-only owner utility.
1. The Knowledge Center uses protected routes for PDF view/download and does not link directly to static PDF paths.
1. KMS v3.0 Phase 3A adds a true engineering handbook with module map, repository architecture, workflows, and developer onboarding.
1. KMS v3.0 Phase 3B adds database architecture, development standards, UI/UX philosophy, product roadmap, and stronger human/AI developer guidance.

Critical Rules

1. Do not touch SaaS flows unless explicitly approved.
1. Do not touch operational logic unless required by the approved task.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change landing page implementation unless explicitly approved.
1. Do not add a KMS regenerate button until explicitly approved.
1. Do not push or commit unless explicitly requested.

KMS Policy

Every implementation must include a Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included docs change, regenerate:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

Development Workflow

1. Read this file first.
2. Read `docs/TIS_MASTER_CONTEXT.md` and `docs/PROJECT_STATE.md`.
3. Read `docs/engineering/README.md`.
4. Read `docs/engineering/DEVELOPMENT_STANDARDS.md`.
5. Read relevant engineering docs, ADRs, module history, and supporting docs.
6. Inspect code before editing.

7. Keep changes scoped.
8. Update KMS docs when meaningful behavior, architecture, product state, module map, repository ownership, data model, design philosophy, roadmap, or workflow changes.
9. Regenerate PDF if included source docs changed.
10. Run validation.
11. Report KIA in final response.

Landing Page Situation

The public landing implementation is in `tis-landing-website/`. Marketing docs live under `docs/marketing/`. Do not modify legacy FastAPI landing files unless explicitly approved.

Next Planned Work

After Phase 2C review, a possible future enhancement is an explicit owner-only regenerate action. It should rebuild the PDF from reviewed Markdown source files only and must not silently rewrite Markdown source docs.

docs/DOCUMENTATION_UPDATE_POLICY.md

TIS Documentation Update Policy

This policy defines the non-negotiable rules for keeping the TIS Knowledge Management System reliable.

Source Of Truth

Markdown files under `docs/` are the source of truth.

The PDF booklet at `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot. It must never be edited manually. It must be regenerated whenever included Markdown source files change.

Required Knowledge Impact Assessment

Every approved implementation must end with a Knowledge Impact Assessment (KIA).

Required final report template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until the KIA is included, even when no documentation changes are needed.

When Documentation Must Be Updated

Update documentation when a task changes:

1. product vision or positioning,
1. architecture or module boundaries,
1. SaaS signup, login, onboarding, billing, payment, or provisioning flows,
1. platform owner or permission behavior,
1. operational workflows such as calendar, planning, timetable, teachers, subjects, or observations,
1. deployment assumptions,
1. branch/release/project status,
1. landing page source-of-truth or visual strategy,
1. roadmap or known issues,
1. documentation system behavior.

Which Docs To Update

Use this guide:

1. `docs/AI_PROJECT_CONTEXT.md`: compact onboarding context for future AI coding conversations; update when the high-level project situation changes.
1. `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.

1. `docs/PROJECT_STATE.md`: current branch strategy, priorities, milestone status, known issues, and next work.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
1. `docs/adr/`: major architectural or product decisions.
1. `docs/history/`: deeper module-specific before/after history.
1. Supporting docs under `docs/marketing/` or other folders when those areas change.

ADR Rule

Create or update an ADR when a change affects a major long-term decision, including identity boundaries, payment architecture, provisioning strategy, deployment architecture, public website architecture, documentation governance, or visual system strategy.

Do not create ADRs for routine bug fixes unless they introduce or reverse a significant decision.

Module History Rule

Update `docs/history/<module>/` when a meaningful module area changes and the previous documented state should be preserved.

`docs/CHANGE_HISTORY.md` remains the chronological summary. Module history stores deeper context.

PDF Regeneration

Regenerate the PDF with:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

The generator must remain dependency-light:

1. use existing `reportlab`,
1. no LaTeX,
1. no Playwright or Chromium,
1. no external network calls,
1. no system font dependency.

Reporting Rule

Every final implementation report must mention:

1. whether knowledge was impacted,
1. which docs changed,
1. whether change history changed,
1. whether an ADR was needed,
1. whether module history changed,
1. whether the PDF was regenerated,
1. whether `AI_PROJECT_CONTEXT.md` changed,
1. validation results.

App Behavior Rule

The application may later detect documentation freshness and expose docs through owner-only protected routes. It must not silently rewrite Markdown source docs. Source docs are edited through reviewed development work.

docs/TIS_MASTER_CONTEXT.md

TIS Master Context

Documentation version: 3.0

Last major context update: 2026-06-26

Product Identity

TIS stands for Teacher Information System. It is a developing SaaS academic operations platform for schools, school groups, academic leaders, supervisors, and platform owners who need one trusted place for academic staffing, teacher records, planning, calendars, observations, branch context, and future intelligence.

TIS is not only a teacher directory. It is intended to become the operational backbone for academic decision-making across a school or multi-branch organization.

Public product presence:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`
1. Operational login: `/login`
1. SaaS signup: `/saas/signup`
1. SaaS login: `/saas/login`
1. SaaS account area: `/saas/account`

Product Vision

TIS helps schools move away from scattered spreadsheets and disconnected operational files. The product vision is to give academic leaders a structured, secure, and tenant-isolated platform where teacher information, teaching loads, staffing needs, observations, calendars, and branch-level context can be managed together.

As the platform matures, TIS should also become the trusted data foundation for AI-assisted academic operations. Future AI features should depend on verified school data, clear permissions, and careful subscription packaging.

Business Goal

The business goal is to establish TIS as a subscription-based SaaS platform for academic operations. The platform should support school onboarding, plan selection, payment, provisioning, account management, and scalable multi-tenant usage.

Near-term business priorities:

1. Convert interested schools from public landing page traffic into SaaS accounts.
1. Support clear onboarding from signup through school setup.
1. Provide reliable billing and payment status visibility.
1. Enable platform owners to manage pending organizations and provisioning.
1. Preserve trust by keeping tenant data isolated and operational flows stable.

Educational Goal

The educational goal is to reduce administrative friction so schools can make better academic decisions. TIS should help leadership teams identify staffing gaps, workload problems, incomplete academic coverage, observation follow-up needs, and calendar conflicts earlier.

The platform should support educational quality by making academic operations easier to see, review, and improve.

Target Customers

Primary customers:

- Private schools
- Multi-branch school groups
- Academic leadership teams
- Principals and vice principals
- Department heads and supervisors
- School operations and HR-adjacent academic staff

Internal platform users:

- Platform Owner
- Platform Co-Owner
- Platform Developer

Tenant users:

- Administrators
- Coordinators
- Supervisors
- Teachers and academic staff, depending on enabled workflows

FastAPI App Architecture

The operational TIS application is a FastAPI application at the repository root. It uses Python, SQLAlchemy, Jinja templates, static assets, and modular routers.

Core app areas:

- `main.py`: primary FastAPI app, route registration, app-level workflows, startup checks, platform console routes, dashboard and configuration flows.
- `models.py`: main SQLAlchemy models.
- `database.py`: database connection/session setup.
- `db_migrations.py`: local schema migration and repair logic.
- `auth.py`: authentication, role normalization, platform identity helpers, permission helpers, session handling.
- `authorization.py`: route permission rules and access-denied response helpers.
- `permission_registry.py`: permission groups, labels, defaults, developer-assignable permissions, and system owner permissions.
- `role_permission_service.py`: role permission persistence and related helpers.

1. `ui_shell.py`: shared application shell context, navigation, visual identity, and page metadata.
1. `routers/`: modular feature routers for users, teachers, subjects, planning, timetable, academic calendar, and observations.
1. `templates/`: Jinja templates for the operational app and SaaS app pages.
1. `static/`: CSS, JavaScript, images, branding assets, and generated public artifacts.
1. `tests/`: pytest coverage for tenant isolation, SaaS phases, platform access, permissions, email, branding, and related workflows.

Important operational route families:

1. `/login`: operational app login.
1. `/dashboard`: tenant operational dashboard.
1. `/platform`: platform console for platform identities.
1. `/system-configuration`: branch, year, branding, role permissions, and configuration workflows.
1. `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, `/observations`: core academic operations.

Next.js Landing Architecture

The public marketing website lives in `tis-landing-website/`. It is separate from the FastAPI operational portal.

Landing architecture:

1. Runtime: Next.js / Node.
1. Public website domain: `https://tisplatform.com`.
1. Local development URL: `http://localhost:3000`.
1. Main page: `tis-landing-website/src/app/page.tsx`.
1. App layout: `tis-landing-website/src/app/layout.tsx`.
1. Global styles: `tis-landing-website/src/app/globals.css`.
1. Logo component: `tis-landing-website/src/components/tis-logo.tsx`.
1. Public assets: `tis-landing-website/public/`.

The application portal remains separate:

1. App domain: `https://app.tisplatform.com`.
1. Runtime: FastAPI / Python with a relational database.

Legacy landing files in the FastAPI app are not the source of truth:

1. `templates/landing.html`
1. `static/landing/landing.css`

Those legacy files must not be modified unless explicitly approved.

Multi-Tenant SaaS Strategy

TIS is designed as a multi-tenant SaaS platform. Tenant isolation is a critical rule. School groups, branches, users, academic years, teachers, planning data, timetable data, observations, and configuration records must remain scoped to the correct organization and branch context.

The platform distinguishes between:

1. Platform identities: platform owners, co-owners, and developers.
1. Tenant identities: users belonging to a school group and branch context.
1. SaaS account identities: accounts that move through signup, onboarding, billing, and provisioning.

Platform users may inspect or switch organization context through controlled platform workflows. Tenant users must remain inside their authorized school, branch, academic year, and permission scope.

Critical tenant strategy:

1. Do not weaken tenant isolation.
1. Do not bypass permission checks.
1. Do not assume a platform identity is a tenant identity.
1. Do not assume a SaaS account is already provisioned into an operational tenant.
1. Keep onboarding, billing, and tenant provisioning as distinct stages.

SaaS Routes And Account Experience

Core SaaS routes:

1. `/saas/signup`: public SaaS account creation.
1. `/saas/login`: SaaS account login.
1. `/saas/account`: SaaS account dashboard.

Related SaaS areas include plan selection, onboarding organization details, contacts, branch setup, academic setup, onboarding review, billing status, checkout summary, checkout return, checkout cancel, sessions, security, and profile pages.

Platform owner SaaS administration exists under `/saas-admin` for pending organizations, payments, and provisioning workflows.

M1-M5 Completed Milestone Summary

M1: Identity and SaaS foundation

1. Established core SaaS account concepts.
1. Added initial signup/login/account flows.
1. Clarified separation between platform, tenant, and SaaS account identities.
1. Added supporting tests for identity and SaaS phase behavior.

M2: Onboarding foundation

1. Added structured onboarding stages for organization information, contacts, branches, academic setup, and review.
1. Improved the path from SaaS signup toward a provisionable organization.
1. Preserved the distinction between pending SaaS organizations and operational tenants.

M3: Billing and plan foundation

1. Added pricing, plan catalog, billing status, checkout summary, return, and cancel flows.
1. Created billing/payment service modules to keep payment logic separate from operational academic logic.
1. Prepared the platform for subscription-based SaaS packaging.

M4: Provisioning foundation

1. Added pending organization and provisioning workflows for platform owners.
1. Added provisioning queue concepts and retry/run operations.
1. Created a controlled path for turning a pending SaaS organization into an operational school context.

M5: Platform access, permissions, and owner controls

1. Strengthened platform identity handling.
1. Added platform owner/developer concepts and owner management controls.
1. Improved permission boundaries for platform and tenant users.
1. Added tests around platform access and role permissions.

Paddle And Payment Architecture Summary

The payment architecture is organized under the `saas/` package.

Key modules:

1. `saas/pricing_service.py`: plan/pricing behavior.
1. `saas/payment_service.py`: payment-related service logic.
1. `saas/paddle_client.py`: Paddle integration boundary.
1. `saas/billing_service.py`: billing status and related account billing behavior.
1. `saas/currency_service.py`: currency-related helpers.
1. `saas/router.py`: SaaS and SaaS admin routes.

Architecture rules:

1. Keep Paddle-specific details behind service/client boundaries.
1. Do not mix payment logic into academic operations.
1. Treat payment status, onboarding status, and provisioning status as related but separate concepts.
1. Use platform owner admin views for payment/provisioning oversight.
1. Avoid changing live SaaS payment flows unless the task explicitly requires it.

Tenant Provisioning Summary

Tenant provisioning turns a pending SaaS organization into an operational TIS school context. Provisioning should create or connect the required organization records, branches, users, and initial tenant setup while preserving auditability and data isolation.

Key provisioning concepts:

1. Pending organization: SaaS onboarding entity not yet fully operational.

1. Provisioning job: controlled action to create or update operational tenant structures.
1. Platform owner review: human oversight before or during provisioning.
1. Retry behavior: failed provisioning work should be recoverable without corrupting tenant data.

Provisioning rules:

1. Do not directly create operational tenant data from public signup without the approved provisioning path.
1. Keep provisioning idempotent where possible.
1. Log or surface errors clearly.
1. Do not merge tenant data across school groups.

Landing Page Strategy

The public landing page exists to explain the product, build trust, capture demand, and route interested schools into SaaS signup or demo request flows.

Source of truth:

1. The public website implementation is `tis-landing-website/`.
1. Marketing content references live in `docs/marketing/`.

Landing priorities:

1. Explain the problem of scattered academic operations.
1. Present TIS as a connected academic operations platform.
1. Show credible platform capabilities.
1. Support early access, demo requests, and signup pathways.
1. Keep the landing page separate from operational app templates.

Landing rule:

1. Do not change landing page design, copy, or architecture during operational backend tasks unless explicitly approved.

Customer Experience Roadmap

Near-term customer experience:

1. Clear signup and login path.
1. Smooth onboarding for organization, contacts, branches, and academic setup.
1. Transparent billing/plan status.
1. Clear provisioning status after checkout or owner review.
1. Reliable operational login once provisioned.

Medium-term customer experience:

1. Better account self-service.
1. Clearer subscription lifecycle.
1. More guided onboarding and implementation support.

1. Improved platform owner visibility into pending organizations and payment status.

Long-term customer experience:

1. AI-assisted academic planning and decision support.
1. Subscription-gated advanced analytics.
1. Intelligent recommendations based on verified tenant data.
1. Richer executive visibility across branches and academic years.

Knowledge Management System

TIS uses a Knowledge Management System (KMS) to preserve source-of-truth documentation, current project state, decision history, module history, and generated snapshots.

KMS source documents:

1. `docs/AI_PROJECT_CONTEXT.md`: first-read compact onboarding context for future Codex and ChatGPT conversations.
1. `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.
1. `docs/PROJECT_STATE.md`: living project state.
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`: mandatory KMS and Knowledge Impact Assessment rules.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
1. `docs/adr/`: Architecture Decision Records.
1. `docs/history/`: module-specific history.
1. `docs/engineering/`: engineering handbook with module map, repository architecture, user/system flows, and developer onboarding.

PDF philosophy:

1. Markdown files are the source of truth.
1. The PDF booklet is a generated snapshot.
1. The PDF must never be edited manually.
1. The PDF must be regenerated when included Markdown source docs change.
1. The PDF should later be served through owner-only protected routes.

Generated KMS artifacts:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`
1. `static/docs/docs_manifest.json`

Owner-only app access:

1. `/platform/knowledge`: read-only Platform Owner Knowledge Center.
1. `/platform/knowledge/booklet`: protected inline PDF view.
1. `/platform/knowledge/booklet/download`: protected PDF download.

The Knowledge Center is protected by the existing Platform Owner access pattern. It is not public, not a landing page, and does not regenerate or rewrite source docs.

Engineering handbook:

1. `docs/engineering/TIS_MODULE_MAP.md` maps product/system modules and guardrails.
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md` explains repository ownership and risky files.
1. `docs/engineering/USER_AND_SYSTEM_FLOWS.md` documents end-to-end customer, SaaS, payment, provisioning, operational, platform owner, KMS, and developer flows.
1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md` explains data areas, ownership boundaries, and tenant isolation rules.
1. `docs/engineering/DEVELOPMENT_STANDARDS.md` defines non-negotiable engineering rules.
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md` defines design direction for operational, platform, SaaS, Knowledge Center, and landing surfaces.
1. `docs/engineering/PRODUCT_ROADMAP.md` records completed, current, next, and future roadmap.

Development Workflow

Default workflow for approved implementation tasks:

1. Inspect the relevant code and docs before editing.
2. Keep changes scoped to the approved task.
3. Preserve tenant isolation, permission checks, SaaS flows, and landing page boundaries.
4. Update tests or add focused tests when behavior changes.
5. Complete the Knowledge Impact Assessment.
6. Update relevant Markdown docs.
7. Update change history, ADRs, module history, and AI project context when needed.
8. Regenerate the documentation PDF if included docs changed.
9. Run reasonable validation.
10. Report code changes, docs changes, KIA, validation, assumptions, and known issues.

Knowledge Impact Assessment Rule

Every approved implementation must:

1. Assess knowledge impact.
2. Update relevant Markdown docs.
3. Update `docs/CHANGE_HISTORY.md` for meaningful changes.
4. Create or update ADRs when major decisions change.
5. Update module history when a module's documented state changes.
6. Update `docs/AI_PROJECT_CONTEXT.md` when high-level AI onboarding context changes.
7. Regenerate the PDF booklet if included docs changed.
8. Mention KIA details in the final report.

A task is not complete until the KIA is assessed.

Required final report KIA template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

The generated booklet output is:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`

Critical Rules Codex Must Follow

1. Do not touch SaaS flows unless the task explicitly requires it.
1. Do not touch operational logic unless the approved task requires it.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass route permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change the landing page design or legacy landing files unless explicitly approved.
1. Do not add KMS regenerate behavior unless explicitly approved.
1. Do not expose the KMS PDF through direct public static links in the app UI.
1. Do not push or commit unless explicitly requested.
1. Prefer conservative, dependency-light automation.
1. Use `reportlab` for the documentation PDF generator.
1. Do not require LaTeX, Playwright, Chromium, external network calls, or system font dependencies for PDF generation.
1. Always include a Knowledge Impact Assessment in implementation final reports.

docs/PROJECT_STATE.md

TIS Project State

Last Updated

Last updated: 2026-06-26

Update this file after every meaningful milestone, active development change, roadmap shift, known issue change, or documentation/KMS change.

Current Branch Strategy

Current working branch assumption: `dev`.

Branch strategy:

1. Development work should happen on `dev` unless the owner explicitly requests another branch.
1. Production/live branch is assumed to be separate from active development.
1. Confirm production branch before any deployment, merge, or production-sensitive change.
1. Do not push, merge, or commit unless explicitly requested.
1. Preserve unrelated local changes.

Production / Live Branch Assumption

The live production branch is assumed to be the branch deployed to the public app environment, while `dev` is the active development branch. This assumption must be confirmed before deployment.

Production domains:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`

Completed Milestones

M1: Identity and SaaS foundation

1. Core SaaS signup/login/account concepts established.
1. Platform, tenant, and SaaS account identities separated.
1. Identity and SaaS phase tests present.

M2: Onboarding foundation

1. SaaS onboarding flow covers organization, contacts, branches, academic setup, and review.
1. Pending organization concept supports pre-provisioning state.

M3: Billing and plan foundation

1. Plan catalog, checkout, billing status, checkout return, and checkout cancel flows exist.
1. Payment and billing code is isolated under `saas/` service modules.

M4: Provisioning foundation

- Platform owner provisioning views and actions exist.
- Pending organization review and provisioning queue behavior exist.
- Provisioning retry/run operations are present.

M5: Platform access and owner controls

- Platform owner and platform developer identities exist.
- Platform console and owner/developer management controls exist.
- Permission registry and platform access tests support this boundary.

Documentation/KMS milestones:

- Phase 1 documentation foundation completed and pushed to `dev`.
- Phase 2A and Phase 2B KMS foundation approved for implementation.
- Phase 2C Platform Owner Knowledge Center completed and pushed to `dev`.
- KMS v3.0 Phase 3A Engineering Handbook approved for implementation.
- KMS v3.0 Phase 3B approved for implementation.

Current Priority

Current priority: upgrade the generated reference booklet into a true TIS Engineering Handbook.

Phase 2A and Phase 2B scope:

- Create documentation update policy.
- Create change history.
- Create ADR foundation and initial accepted ADRs.
- Create module history foundation.
- Create AI project context.
- Update master context, project state, and documentation index.
- Update PDF generator to include KMS docs and manifest metadata.
- Regenerate `static/docs/TIS_Project_Reference_Booklet.pdf`.

Phase 2C completed scope:

- Added read-only `knowledge_service.py` as the single KMS app access layer.
- Added owner-protected `/platform/knowledge` page.
- Added owner-protected PDF view/download routes.
- Added an owner-only Platform Console card.
- Added platform knowledge module history.
- Regenerated the PDF and manifest after documentation updates.

Still out of scope:

- Regenerate button.

- Additional app routes beyond the approved read-only Knowledge Center routes.
- `ui_shell.py` and `authorization.py` changes unless separately approved.
- SaaS flows.
- Operational logic.
- Database, migrations, or `tis.db`.
- Landing page implementation.

KMS v3.0 Phase 3A scope:

- Add complete TIS module map.
- Add repository architecture map.
- Add end-to-end user/system workflows.
- Add clear AI/human developer onboarding structure.
- Update generator to include engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3B scope:

- Add database architecture overview.
- Add development standards and non-negotiable rules.
- Add UI/UX and design philosophy.
- Add product roadmap.
- Strengthen AI/human developer onboarding guidance.
- Update generator to include the new engineering docs.
- Regenerate the PDF and manifest.

Current Known Issues

Known issues and watch points:

- KMS policy depends on future developers and AI agents consistently completing the Knowledge Impact Assessment.
- Generated PDF can become stale if included Markdown docs change without regeneration.
- The owner-only Knowledge Center is implemented as read-only; there is no regenerate button yet.
- Public static storage is not sufficient access control for sensitive docs; Phase 2C should serve docs through protected owner-only routes.
- Render deployment constraints should continue to guide dependency choices.
- Broad filesystem scans may warn about `tis_scope_test_5i3yf0h5/` access denial.

Next Planned Work

Next planned work after Phase 2C review:

- Review KMS v3.0 Phase 3B handbook coverage, PDF readability, and manifest inclusion.
- Approve corrections if needed.

1. Later consider an explicit owner-only regenerate workflow.

Landing Page Baseline Situation

The public landing page source of truth is:

1. `tis-landing-website/`

Marketing docs:

1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`

Relevant ADRs:

1. `docs/adr/0001-separate-nextjs-landing-website.md`
1. `docs/adr/0007-landing-page-visual-system-strategy.md`

Legacy FastAPI landing files are not the current public website source of truth:

1. `templates/landing.html`
1. `static/landing/landing.css`

Do not modify landing page design, landing copy, or legacy landing files unless explicitly approved.

Knowledge Update Policy

Every approved implementation must complete the KIA:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included docs change, regenerate:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`

Scope Guardrails

1. Do not touch SaaS flows unless explicitly approved.
1. Do not touch operational logic unless required by the approved task.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not change the landing page unless explicitly approved.
1. Do not add Platform Owner Knowledge Center routes until reviewed and approved.
1. Do not add a KMS regenerate button until separately approved.
1. Do not implement Phase 3B until reviewed and approved.
1. Do not implement Phase 3C until reviewed and approved.
1. Do not commit or push unless explicitly requested.

docs/CHANGE_HISTORY.md

TIS Change History

This file is the chronological summary of meaningful TIS changes. It does not replace module history under `docs/history/`; it gives reviewers, developers, Codex, and ChatGPT a fast timeline of what changed and why.

Newest entries should be added first.

Entry Template

```
## YYYY-MM-DD - Short Change Title

Area/module:
Previous state:
New state:
Reason:
Files changed:
Documentation updated:
PDF regenerated:
AI project context updated:
Reviewer/approval notes:
```

2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3A added module map, repository architecture, user/system flows, and onboarding structure. The handbook still needed database architecture, development standards, UI/UX philosophy, roadmap, and stronger human/AI guidance.

New state: TIS now has Phase 3B engineering docs for database architecture, development standards, UI/UX design philosophy, and product roadmap. Core KMS docs and AI onboarding guidance reference these layers, and the PDF generator includes them.

Reason: Make the generated booklet more useful for new senior developers, Codex conversations, ChatGPT conversations, and future technical reviewers.

Files changed:

1 `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`

1 `docs/engineering/DEVELOPMENT_STANDARDS.md`

1 `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`

1 `docs/engineering/PRODUCT_ROADMAP.md`

1 `docs/engineering/README.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

1 `docs/CHANGE_HISTORY.md`

1 `scripts/generate_docs_pdf.py`

1 `static/docs/TIS_Project_Reference_Booklet.pdf`

```
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Engineering Handbook

Area/module: Knowledge Management System and engineering onboarding

Previous state: The generated booklet included KMS source documents, ADRs, module history, AI context, and the Knowledge Center foundation, but it did not fully onboard a new human developer or future Codex/ChatGPT conversation into TIS modules, repository architecture, and end-to-end flows.

New state: TIS now has an engineering handbook layer with a complete module map, repository architecture guide, user/system flow guide, and engineering onboarding index. The PDF generator includes these docs and emits documentation version 3.0.

Reason: Make the generated booklet a true TIS Engineering Handbook rather than only a documentation bundle.

Files changed:

```
docs/engineering/README.md
```

```
docs/engineering/TIS_MODULE_MAP.md
```

```
docs/engineering/REPOSITORY_ARCHITECTURE.md
```

```
docs/engineering/USER_AND_SYSTEM_FLOWS.md
```

```
docs/TIS_MASTER_CONTEXT.md
```

```
docs/AI_PROJECT_CONTEXT.md
```

```
docs/PROJECT_STATE.md
```

```
docs/README.md
```

```
docs/CHANGE_HISTORY.md
```

```
scripts/generate_docs_pdf.py
```

```
static/docs/TIS_Project_Reference_Booklet.pdf
```

```
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added Platform Owner Knowledge Center

Area/module: Platform Knowledge Center and KMS access

Previous state: TIS had KMS source docs, ADRs, module history, a generated PDF booklet, and a manifest, but no protected in-app owner page for KMS status or booklet access.

New state: TIS now has a read-only Platform Owner Knowledge Center with KMS health score, manifest metadata, freshness detection, source document status, coverage checks, latest change-history entries, ADR list, module history areas, KIA checklist, and protected PDF view/download routes.

Reason: Platform owners need an internal utility for verifying KMS health and accessing the generated PDF without exposing direct public static links.

Files changed:

- `knowledge_service.py`
- `main.py`
- `templates/platform_knowledge_center.html`
- `templates/platform_console.html`
- `scripts/generate_docs_pdf.py`
- `docs/TIS_MASTER_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/README.md`
- `docs/CHANGE_HISTORY.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/history/platform-knowledge/README.md`
- `docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md`
- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2C only. Regenerate button, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

2026-06-26 - Established Knowledge Management System Foundation

Area/module: Documentation and project knowledge management

Previous state: TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal change history, ADR system, module history foundation, KMS policy, manifest, or compact AI onboarding file.

New state: TIS now has a Knowledge Management System foundation with chronological change history, documentation update policy, ADR structure and initial accepted ADRs, module history folders, AI project context, updated source docs, and an expanded PDF generator.

Reason: Preserve project knowledge for future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and technical reviewers.

Files changed:

```
docs/CHANGE_HISTORY.md
docs/DOCUMENTATION_UPDATE_POLICY.md
docs/AI_PROJECT_CONTEXT.md
docs/adr/README.md
docs/adr/0001-separate-nextjs-landing-website.md
docs/adr/0002-separate-saas-identity-and-operational-users.md
docs/adr/0003-paddle-payment-architecture.md
docs/adr/0004-webhook-only-payment-confirmation.md
docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
docs/adr/0006-documentation-as-source-knowledge-management-system.md
docs/adr/0007-landing-page-visual-system-strategy.md
docs/history/README.md
docs/history/*/README.md
docs/history/provisioning/2026-06-26-kms-foundation.md
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only. Platform Owner Knowledge Center, app routes, SaaS flows, database, migrations, landing page implementation, commits, and pushes remain out of scope.

docs/adr/README.md

Architecture Decision Records

ADRs record major TIS architectural and product decisions. They explain why the system is shaped the way it is.

ADR Rules

1. Create an ADR for major decisions affecting architecture, identity, SaaS, payment, provisioning, deployment, documentation governance, or long-term product boundaries.
1. Do not use ADRs for ordinary bug fixes.
1. If a decision is replaced, mark the older ADR as Superseded and link the new ADR.
1. ADRs complement `docs/TIS_MASTER_CONTEXT.md`; they do not replace it.

Status Values

1. Proposed
1. Accepted
1. Superseded
1. Deprecated

ADR Index

1. `0001-separate-nextjs-landing-website.md`
1. `0002-separate-saas-identity-and-operational-users.md`
1. `0003-paddle-payment-architecture.md`
1. `0004-webhook-only-payment-confirmation.md`
1. `0005-delayed-tenant-provisioning-after-verified-payment.md`
1. `0006-documentation-as-source-knowledge-management-system.md`
1. `0007-landing-page-visual-system-strategy.md`

docs/engineering/README.md

TIS Engineering Handbook

This folder turns the TIS KMS into a practical engineering handbook. It is intended for new human developers, future Codex conversations, future ChatGPT conversations, project owners, platform owners, and reviewers.

Read First

Recommended onboarding order:

1. `docs/AI_PROJECT_CONTEXT.md`
2. `docs/TIS_MASTER_CONTEXT.md`
3. `docs/PROJECT_STATE.md`
4. `docs/engineering/TIS_MODULE_MAP.md`
5. `docs/engineering/REPOSITORY_ARCHITECTURE.md`
6. `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
7. Relevant ADRs under `docs/adr/`
8. Relevant module history under `docs/history/`

Engineering Documents

1. `TIS_MODULE_MAP.md`: product and system module map with purpose, files, maturity, docs, risks, and guardrails.
1. `REPOSITORY_ARCHITECTURE.md`: repository structure and responsibilities.
1. `USER_AND_SYSTEM_FLOWS.md`: end-to-end public, SaaS, payment, provisioning, operational, platform owner, and KMS flows.
1. `DATABASE_ARCHITECTURE_OVERVIEW.md`: high-level data model relationships and isolation boundaries.
1. `DEVELOPMENT_STANDARDS.md`: non-negotiable engineering rules for humans and AI assistants.
1. `UI_UX_DESIGN_PHILOSOPHY.md`: design principles for operational app, platform owner tools, SaaS onboarding, Knowledge Center, and landing website.
1. `PRODUCT_ROADMAP.md`: completed, current, next, and future roadmap.

Before Coding

Before changing code:

1. confirm the approved scope,
1. inspect affected files and tests,
1. read relevant ADRs,
1. read relevant module history,
1. preserve tenant isolation,
1. preserve identity boundaries,
1. avoid touching SaaS, landing, database, or migrations unless explicitly approved.

After Coding

Every implementation must complete the Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included source docs changed, regenerate:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

docs/engineering/TIS_MODULE_MAP.md

TIS Module Map

This map describes the known TIS modules, where they live, their maturity, related docs/ADRs, and the guardrails developers must respect.

Platform Owner

Purpose: Own global TIS platform operations, owner/co-owner controls, platform developer accounts, cross-organization oversight, and protected KMS access.

Main files/folders:

- `auth.py`
- `main.py`
- `templates/platform_console.html`
- `templates/platform_knowledge_center.html`
- `knowledge_service.py`
- `permission_registry.py`

Maturity/status: Implemented for owner identity, platform console, developer/co-owner controls, and read-only Knowledge Center.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not treat platform developers as owners.
- Do not expose owner utilities to tenant users.
- Reuse `auth.is_platform_owner(...)` and existing owner access helpers.

Platform Console

Purpose: Allow platform identities to inspect organizations, switch context, and manage platform owner/developer controls.

Main files/folders:

- `main.py`
- `templates/platform_console.html`
- `ui_shell.py`

Maturity/status: Implemented and active.

Related docs/ADRs:

- `docs/PROJECT_STATE.md`

Risks/guardrails:

- Context switching must not weaken tenant isolation.
- Owner-only management controls must remain owner-only.

Knowledge Center

Purpose: Show KMS health, source coverage, PDF freshness, ADRs, module history, change history, and KIA policy to platform owners.

Main files/folders:

- `knowledge_service.py`
- `main.py`
- `templates/platform_knowledge_center.html`
- `static/docs/docs_manifest.json`
- `static/docs/TIS_Project_Reference_Booklet.pdf`

Maturity/status: Read-only Phase 2C implementation complete. No regenerate button.

Related docs/ADRs:

- `docs/adr/0006-documentation-as-source-knowledge-management-system.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not link directly to `/static/docs/...` from UI.
- Do not let the app rewrite Markdown source docs.
- Do not add regenerate behavior without approval.

SaaS Account System

Purpose: Handle public SaaS signup/login/account profile, sessions, security, billing visibility, and onboarding status.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `templates/saas/`

Maturity/status: M1-M5 foundation implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`
- `docs/history/saas-onboarding/`

Risks/guardrails:

- Do not merge SaaS account identity with operational tenant users.

- Do not bypass verification/security/session boundaries.

SaaS Onboarding

Purpose: Collect organization, contact, branch, academic setup, and review data before provisioning.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `templates/saas/onboarding_*.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/history/saas-onboarding/`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Pending onboarding data is not the same as operational tenant data.
- Do not create live tenant records directly from public forms.

Pending Organizations

Purpose: Represent SaaS organizations waiting for review, payment readiness, or provisioning.

Main files/folders:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `templates/saas/admin_pending_organizations.html`
- `templates/saas/admin_pending_organization_detail.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

Risks/guardrails:

- Keep pending state separate from provisioned tenant state.
- Preserve platform owner review and auditability.

Plans And Pricing

Purpose: Define SaaS plan catalog and pricing behavior.

Main files/folders:

- `saas/pricing_service.py`

- `saas/router.py`
- `templates/saas/plan_catalog.html`
- `templates/saas/plan_selection.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/history/subscriptions/`
- `docs/adr/0003-paddle-payment-architecture.md`

Risks/guardrails:

- Plan changes must update docs and change history.
- Do not hard-code provider behavior outside service boundaries.

Paddle / Payment

Purpose: Integrate subscription checkout, payment state, billing status, and provider events.

Main files/folders:

- `saas/paddle_client.py`
- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/currency_service.py`
- `saas/router.py`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`
- `docs/history/subscriptions/`

Risks/guardrails:

- Webhook-confirmed payment state is authoritative.
- Checkout return alone must not trigger verified payment/provisioning.

Provisioning

Purpose: Convert verified/approved pending organizations into operational tenant structures.

Main files/folders:

- `saas/provisioning_service.py`
- `saas/router.py`
- `templates/saas/admin_provisioning.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- `docs/history/provisioning/`

Risks/guardrails:

- Keep provisioning recoverable and reviewable.
- Do not merge tenants or create records in the wrong school group.

Operational Login

Purpose: Authenticate operational users and route them into platform or tenant context.

Main files/folders:

- `main.py`
- `auth.py`
- `templates/login.html` if present through root templates

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Platform users and tenant users follow different context behavior.
- Preserve permission and active-user checks.

Dashboard

Purpose: Give tenant users operational visibility into staffing, planning, reports, and current school context.

Main files/folders:

- `main.py`
- `templates/dashboard.html`
- related report/export helpers

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Dashboard data must remain scoped to tenant/branch/year context.

Organizations / School Groups

Purpose: Represent tenant organizations and top-level school group context.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_schools.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Tenant isolation starts at school group boundaries.

Branches

Purpose: Represent campuses/branches inside a school group.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_branches.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Branch scope affects teachers, users, planning, timetable, calendar, branding, and reports.

Academic Years

Purpose: Scope operational data by academic year and active-year context.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_years.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Do not mix data across academic years.

Users And Roles

Purpose: Manage tenant users, platform users, roles, profile data, active status, and branch/year context.

Main files/folders:

- `auth.py`
- `routers/users.py`
- `models.py`
- `templates/users.html`
- `templates/edit_user.html`

Maturity/status: Implemented with platform identity refinements.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Do not assign owner controls to developers or tenant users.
- Preserve role normalization.

Permissions

Purpose: Control route/module actions by permission key and role package.

Main files/folders:

- `authorization.py`
- `permission_registry.py`
- `role_permission_service.py`
- `auth.py`
- `templates/system_configuration_role_permissions.html`

Maturity/status: Implemented and critical.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Do not bypass `authorization.enforce_route_permission`.
- Do not rely only on UI hiding for protected actions.

Teachers

Purpose: Manage teacher records, qualifications, capacity, workloads, and profile-related academic staffing data.

Main files/folders:

```
routers/teachers.py
teacher_qualifications.py
teacher_capacity.py
models.py
templates/teachers.html
templates/edit_teacher.html
```

Maturity/status: Implemented and central to operational value.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Teacher data must remain tenant/branch/year scoped.
```

Subjects

Purpose: Manage subject catalog, colors, qualifications, and planning/timetable relationships.

Main files/folders:

```
routers/subjects.py
subject_colors.py
models.py
templates/subjects.html
templates/edit_subject.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Subject changes can affect planning, timetable, teacher qualifications, and reports.
```

Sections

Purpose: Represent class/section structures used by planning and timetabling.

Main files/folders:

```
routers/planning.py
models.py
templates/planning.html
```

Maturity/status: Implemented through planning workflows.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Section changes can affect timetable allocation and workload reporting.
```

Workforce Planning

Purpose: Plan assignments, homeroom ownership, workloads, subject coverage, and staffing needs.

Main files/folders:

```
routers/planning.py
teacher_capacity.py
homeroom_defaults.py
templates/planning.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Planning logic must preserve branch/year scope and avoid accidental cross-year copies.
```

Timetable

Purpose: Place planned lessons into weekly timetable grids and exports.

Main files/folders:

```
routers/timetable.py
timetable_logic.py
templates/timetable.html
templates/system_configuration_timetable.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Timetable rules interact with planning, sections, subjects, and teacher capacity.
```

Academic Calendar

Purpose: Manage academic events, responsibilities, dates, exports, and branch/year scoped calendar views.

Main files/folders:

```
routers/academic_calendar.py
```

- `templates/academic_calendar.html`
- `templates/system_configuration_calendar.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/history/academic-calendar/`

Risks/guardrails:

- Calendar permissions and branch/year scoping must remain intact.

Observations

Purpose: Support teacher observations, feedback, evidence, scoring, history, and supervision records.

Main files/folders:

- `routers/observations.py`
- `templates/observations.html`
- `templates/observation_form.html`
- `templates/observation_detail.html`
- `templates/observation_history.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Observation records are sensitive and must remain tenant scoped.

Reports / Dashboards

Purpose: Expose operational summaries, exports, allocation reports, and decision visibility.

Main files/folders:

- `main.py`
- report/export helper code
- `templates/dashboard.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Report data must be permission-checked and tenant scoped.

Branding / Design Settings

Purpose: Manage organization logos, design settings, visual shell behavior, and platform visual controls.

Main files/folders:

- `branding_storage.py`
- `design_tokens.py`
- `visual_design.py`
- `ui_shell.py`
- `static/css/branding.css`
- `static/css/design-studio.css`
- `templates/system_configuration_logos.html`
- `templates/system_configuration_design.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/adr/0007-landing-page-visual-system-strategy.md`

Risks/guardrails:

- Do not confuse operational app branding with public landing website design.
- Protect uploaded/owned assets.

Landing Website

Purpose: Public marketing and conversion surface for TIS.

Main files/folders:

- `tis-landing-website/`
- `docs/marketing/`

Maturity/status: Implemented as separate Next.js app.

Related docs/ADRs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/marketing/landing_page_source_of_truth.md`

Risks/guardrails:

- Do not modify landing code during operational/backend tasks unless explicitly approved.
- Legacy FastAPI landing files are not the public source of truth.

AI Future Roadmap

Purpose: Future subscription-gated AI-assisted planning, analytics, assessment generation, recommendations, and decision support.

Main files/folders:

- No dedicated AI production module yet.
- Future work should be documented before implementation.

Maturity/status: Roadmap / future.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`
- future ADRs required before major AI architecture decisions.

Risks/guardrails:

- AI features must use verified tenant data and preserve privacy, permissions, and subscription boundaries.

docs/engineering/REPOSITORY_ARCHITECTURE.md

TIS Repository Architecture

This document explains the main repository areas, what each owns, and what must not be changed casually.

Root FastAPI Application

`main.py`

Responsibility: Primary FastAPI app, route registration, many app-level workflows, middleware, startup checks, platform console routes, dashboards, exports, system configuration, scope switching, and operational pages.

Do not change casually:

1. authentication/session flow,
1. platform owner access checks,
1. tenant scope handling,
1. route behavior shared by many modules,
1. startup/schema repair behavior.

`auth.py`

Responsibility: Password hashing/verification, session cookies, current-user lookup, role normalization, platform identity helpers, permission helpers, email verification helpers, and tenant/platform identity boundaries.

Do not change casually:

1. `is_platform_owner`, `is_platform_user`, `is_platform_developer`,
1. role normalization,
1. session cookie behavior,
1. permission helper semantics.

`authorization.py`

Responsibility: Protected route rules, permission enforcement middleware integration, access-denied responses.

Do not change casually:

1. route permission mapping,
1. public path patterns,
1. permission matching semantics.

`database.py`

Responsibility: Database engine/session setup.

Do not change casually:

1. database URL handling,
1. session creation behavior,

1 engine configuration.

models.py

Responsibility: Primary operational SQLAlchemy models for users, school groups, branches, teachers, planning, timetable, observations, configuration, and related app data.

Do not change casually:

1 tenant ownership fields,

1 platform user fields,

1 relationships used by existing workflows,

1 schema without matching migration/repair strategy.

db_migrations.py

Responsibility: Local schema migration and repair logic used by the app.

Do not change casually:

1 migration ordering,

1 destructive schema operations,

1 production-sensitive repair behavior.

permission_registry.py

Responsibility: Permission groups, labels, default role permissions, system owner permissions, developer-assignable permission boundaries.

Do not change casually:

1 owner/developer permissions,

1 defaults for managed roles,

1 permission keys referenced by routes/templates/tests.

role_permission_service.py

Responsibility: Persistence and service helpers for role permission rows.

Do not change casually:

1 global vs school-scoped permission behavior,

1 role permission constraints.

ui_shell.py

Responsibility: Shared application shell, navigation, page metadata, scoped organization/year context display, logos, visual design CSS, and permission-based nav generation.

Do not change casually:

1 navigation visibility,

1 platform-vs-tenant shell behavior,

- design-studio gating,
- scope display.

Feature Routers: `routers/`

Responsibility: Modular operational route handlers:

- `routers/users.py`
- `routers/teachers.py`
- `routers/subjects.py`
- `routers/planning.py`
- `routers/timetable.py`
- `routers/academic_calendar.py`
- `routers/observations.py`

Do not change casually:

- tenant/branch/year scoping,
- permission checks,
- import/export behavior,
- bulk operations.

SaaS Package: `saas/`

Responsibility: Public SaaS account flows, onboarding, plans, billing, Paddle integration, pending organizations, and provisioning.

Key files:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `saas/pricing_service.py`
- `saas/payment_service.py`
- `saas/paddle_client.py`
- `saas/billing_service.py`
- `saas/provisioning_service.py`
- `saas/currency_service.py`
- `saas/oauth.py`

Do not change casually:

- identity separation,
- payment confirmation rules,
- provisioning readiness,

1. Paddle/webhook boundaries,
1. public signup/onboarding state transitions.

Templates: `templates/`

Responsibility: Jinja templates for operational app, platform console, Knowledge Center, SaaS pages, system configuration, and feature workflows.

Do not change casually:

1. form action routes,
1. hidden scope fields,
1. permission-dependent controls,
1. app shell extension patterns.

Static Assets: `static/`

Responsibility: Operational CSS/JS, images, branding assets, generated documentation PDF and manifest.

Important:

1. `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot.
1. `static/docs/docs_manifest.json` is generated metadata.

Do not change casually:

1. generated docs manually,
1. shared CSS without checking all templates,
1. protected document access assumptions.

Tests: `tests/`

Responsibility: Regression coverage for SaaS phases, tenant isolation, platform access, permissions, email, branding, and critical workflows.

Do not change casually:

1. tests should follow behavior, not hide regressions.
1. update tests when behavior intentionally changes.

Docs: `docs/`

Responsibility: KMS source of truth, engineering handbook, ADRs, change history, module history, AI context, marketing docs.

Do not change casually:

1. historical records should preserve old/new state.
1. update docs through the KIA process.

Scripts: `scripts/`

Responsibility: Maintenance scripts such as `scripts/generate_docs_pdf.py`.

Do not change casually:

1. PDF generator dependency assumptions,
1. source list behavior,
1. manifest metadata.

Next.js Landing Website: `tis-landing-website/`

Responsibility: Public marketing website at `https://tisplatform.com`.

Do not change casually:

1. landing implementation during backend tasks,
1. visual system without approval,
1. public assets/copy without checking marketing docs and ADRs.

docs/engineering/USER_AND_SYSTEM_FLOWS.md

TIS User And System Flows

This document describes the major end-to-end flows a developer must understand before changing TIS.

Public Customer Flow

Flow:

1. Public visitor opens `https://tisplatform.com`.
2. Visitor clicks a signup/get-started path.
3. Visitor reaches SaaS signup at `/saas/signup`.
4. SaaS account is created.
5. Email verification is completed when required.
6. User signs into SaaS account through `/saas/login`.
7. User enters `/saas/account`.
8. User completes organization onboarding:
    1. organization details,
    1. contacts,
    1. branches,
    1. academic setup,
    1. review.
    1. User selects a plan.
    2. User enters checkout.
    3. Paddle handles payment.
    4. Return/cancel page informs the user of checkout navigation result.
    5. Paddle webhook confirms payment.
    6. Local payment/billing state is updated.
    7. Pending organization becomes ready for provisioning.
    8. Platform owner reviews/runs provisioning.
    9. Operational tenant structures are created.
    10. Operational login becomes available through `/login`.

Guardrails:

1. Checkout return is not authoritative payment confirmation.
1. Public signup must not directly create operational tenant data.
1. Provisioning should occur only through the approved flow.

SaaS Identity Flow

Flow:

1. User signs up through `/saas/signup`.
2. SaaS account/session records are created.
3. User signs in through `/saas/login`.
4. Account dashboard is available at `/saas/account`.
5. User can access profile, sessions, security, billing status, and onboarding state.

Important distinction:

1. SaaS account identity is not the same as operational tenant user identity.
1. Platform identity is also separate.

Guardrails:

1. Do not merge SaaS accounts with operational users unless an approved provisioning flow creates the needed operational records.
1. Keep SaaS authentication and operational authentication boundaries clear.

Payment Flow

Flow:

1. User selects a plan.
2. App creates or references a checkout session.
3. User goes to Paddle checkout.
4. User returns through checkout return or cancel pages.
5. Paddle sends webhook events.
6. Webhook-confirmed payment updates local payment/billing state.
7. Verified payment can make a pending organization ready for provisioning.

Guardrails:

1. Webhook confirmation is authoritative.
1. Do not use return-page navigation as proof of payment.
1. Keep Paddle-specific details inside `saas/` payment/client service boundaries.

Provisioning Flow

Flow:

1. Pending organization has completed required onboarding.
2. Payment is verified or owner-approved readiness is satisfied.
3. Provisioning job is queued or run.
4. Operational records are created or connected:

1. school group,
1. branch,
1. academic year,
1. initial operational user,
1. permissions/role context,
1. required setup defaults.

1. Provisioning status is updated.
2. Activation/access email may be sent.
3. User enters operational portal through `/login`.

Guardrails:

1. Keep provisioning idempotent where possible.
1. Do not mix school groups.
1. Do not skip platform owner visibility.

Operational Login Flow

Flow:

1. User opens `/login`.
2. Credentials are verified.
3. Active-user status is checked.
4. Platform users route toward platform context.
5. Tenant users receive branch/year scope.
6. Session and scope cookies are set.
7. User lands on `/platform` or `/dashboard`.
8. Middleware enforces route permissions.

Guardrails:

1. Preserve platform vs tenant branching.
1. Preserve idle timeout behavior for tenant users.
1. Do not bypass permission middleware.

Platform Owner Flow

Flow:

1. Platform owner logs in through `/login`.
2. Owner lands in platform context.
3. Owner uses `/platform` for organization context and owner/developer controls.
4. Owner uses SaaS admin pages for pending organizations, payments, and provisioning.

5. Owner uses `/platform/knowledge` to review KMS health.
6. Owner views/downloads the PDF through protected routes:

- `/platform/knowledge/booklet`
- `/platform/knowledge/booklet/download`

Guardrails:

- Platform developers are not owners.
- Owner-only pages must use existing owner access helpers.
- Do not expose KMS PDF through direct static links.

Knowledge Management Flow

Flow:

1. Developer or Codex reads `docs/AI_PROJECT_CONTEXT.md`.
2. Developer reads master context, project state, engineering docs, relevant ADRs, and module history.
3. Approved change is implemented.
4. Knowledge Impact Assessment is completed.
5. Relevant docs are updated:

- master context,
- project state,
- change history,
- ADRs if needed,
- module history if needed,
- AI project context if needed,
- engineering docs if architecture/module/flow understanding changed.

1. PDF generator runs.
2. PDF snapshot and manifest are regenerated.
3. Knowledge Center checks manifest freshness and health.

Guardrails:

- Markdown remains source of truth.
- PDF is generated and must not be edited manually.
- App must not silently rewrite source docs.
- Regenerate button is not implemented yet.

Human / AI Developer Onboarding Flow

Flow:

1. Read `docs/AI_PROJECT_CONTEXT.md`.
2. Read `docs/README.md`.
3. Read `docs/TIS_MASTER_CONTEXT.md`.
4. Read `docs/PROJECT_STATE.md`.
5. Read `docs/engineering/TIS_MODULE_MAP.md`.
6. Read `docs/engineering/REPOSITORY_ARCHITECTURE.md`.
7. Read `docs/engineering/USER_AND_SYSTEM_FLOWS.md`.
8. Read relevant ADRs and module history.
9. Inspect code with `rg` before editing.
10. Make scoped changes only.
11. Update KMS docs and regenerate the PDF if needed.
12. Report the KIA.

Before coding, inspect:

1. affected routes,
1. models and scope fields,
1. permission rules,
1. templates/forms,
1. tests for the touched module,
1. related docs/ADRs/history.

After coding, update:

1. `docs/CHANGE_HISTORY.md` for meaningful changes,
1. module history for area-specific changes,
1. ADRs for major decisions,
1. engineering docs when module maps, architecture, or flows change,
1. AI context when onboarding truth changes,
1. project state when priority/status changes.

docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

TIS Database Architecture Overview

This document explains the conceptual TIS data model. It is intentionally high level and does not list every field. Use `models.py`, `saas/models.py`, and tests for exact implementation details.

Core Boundary Principle

TIS has three identity/data worlds that must never be casually mixed:

1. Platform data: platform owners, co-owners, developers, platform permissions, and cross-organization oversight.
1. SaaS account data: public signup, onboarding, billing, pending organizations, and payment/provisioning readiness.
1. Operational tenant data: provisioned school groups, branches, academic years, users, teachers, subjects, planning, timetable, calendar, and observations.

The safest mental model:

```
Platform Owner oversees many organizations.
SaaS account prepares an organization for subscription/provisioning.
Operational tenant data belongs to one provisioned school group/branch/year context.
```

Platform Identities

Represents: Platform Owner, Co-Owner, and Platform Developer identities.

Ownership boundary: Platform identities can operate outside ordinary tenant scope only through approved platform workflows.

Must never be mixed:

1. Platform Developer must not become Platform Owner through permission drift.
1. Platform users must not be treated as normal tenant users unless an explicit context workflow establishes scope.

Related files:

1. `auth.py`
1. `models.py`
1. `permission_registry.py`
1. `main.py`

SaaS Accounts

Represents: Public SaaS signup/login/account identities used before or alongside operational tenant access.

Ownership boundary: SaaS accounts own onboarding and billing/account state, not operational school records directly.

Must never be mixed:

1. SaaS account identity is not operational user identity.
1. SaaS account creation must not directly create live tenant data.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Pending Organizations

Represents: An organization moving through SaaS onboarding before operational provisioning.

Ownership boundary: Pending organization data belongs to the SaaS onboarding/provisioning pipeline.

Must never be mixed:

- Pending organization data is not the live school group until provisioning creates or connects operational records.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/provisioning_service.py`

Payment Records

Represents: Payment, billing, checkout, and provider-confirmed subscription state.

Ownership boundary: Payment state belongs to SaaS billing/subscription logic and should not be embedded directly into academic modules.

Must never be mixed:

- Checkout return navigation must not be treated as verified payment.
- Payment/provider details must not leak into operational teacher/planning/calendar logic.

Related files:

- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/paddle_client.py`
- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`

Provisioning Jobs

Represents: Controlled work that turns a ready pending organization into operational school structures.

Ownership boundary: Provisioning bridges SaaS data and operational tenant data, but only through explicit platform-owner-visible workflows.

Must never be mixed:

- Do not directly create operational tenant records from public signup or checkout return pages.

1. Do not create records under the wrong school group.

Related files:

1. `saas/provisioning_service.py`
1. `saas/router.py`
1. `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

School Groups / Organizations

Represents: The top-level operational tenant boundary.

Ownership boundary: Most tenant operational data belongs to a school group directly or through branch/year relationships.

Must never be mixed:

1. Data from one school group must not appear in another school group's operational workflows.

Related files:

1. `models.py`
1. `main.py`

Branches

Represents: Campuses or branches inside a school group.

Ownership boundary: Branches scope users, teachers, planning, timetable, academic calendar, branding, and reports.

Must never be mixed:

1. Branch-scoped data must not silently cross campuses.
1. Platform context switching must remain explicit.

Related files:

1. `models.py`
1. `main.py`
1. `ui_shell.py`

Academic Years

Represents: The academic period used to scope operational records.

Ownership boundary: Planning, timetable, subjects, calendar, and related data may depend on active academic year.

Must never be mixed:

1. Do not copy or read current-year data into another year unless the workflow explicitly does so.

Related files:

1. `models.py`
1. `main.py`

- `year_copy.py`

Operational Users

Represents: Users who work inside an operational school group/branch/year context.

Ownership boundary: Operational users belong to tenant structures and are governed by roles, permissions, branch scope, and active status.

Must never be mixed:

- Operational users are not SaaS accounts by default.
- Tenant users should not gain platform owner access.

Related files:

- `models.py`
- `auth.py`
- `routers/users.py`

Roles And Permissions

Represents: Role packages, permission keys, platform developer permissions, and route/action authorization.

Ownership boundary: Permissions decide what a user can view or change in the current scope.

Must never be mixed:

- UI hiding is not enough; protected actions need route/service checks.
- Owner controls must remain outside developer-assignable permission drift.

Related files:

- `permission_registry.py`
- `role_permission_service.py`
- `authorization.py`
- `auth.py`

Teachers

Represents: Teacher records, qualifications, capacity, workload, and staffing-relevant academic data.

Ownership boundary: Teacher records are tenant/branch/year-sensitive operational data.

Must never be mixed:

- Teacher data from one branch or school group must not appear in another tenant's planning/reporting.

Related files:

- `routers/teachers.py`
- `teacher_qualifications.py`
- `teacher_capacity.py`

- `models.py`

Subjects

Represents: Subject catalog, colors, requirements, qualification relationships, planning and timetable dependencies.

Ownership boundary: Subjects are academic configuration data inside tenant/year context.

Must never be mixed:

- Subject changes can affect planning, timetable, teacher matching, and reports; update all affected docs/tests.

Related files:

- `routers/subjects.py`
- `subject_colors.py`
- `models.py`

Sections / Classes

Represents: Class sections used by planning and timetabling.

Ownership boundary: Sections belong to operational branch/year structures.

Must never be mixed:

- Section structures should not cross branch/year boundaries accidentally.

Related files:

- `routers/planning.py`
- `models.py`

Workforce Planning

Represents: Teacher assignments, homeroom ownership, capacity, workload, subject coverage, and staffing needs.

Ownership boundary: Planning is operational data tied to school group, branch, academic year, teachers, subjects, and sections.

Must never be mixed:

- Planning changes can affect dashboards, reports, timetable, and staffing decisions.

Related files:

- `routers/planning.py`
- `teacher_capacity.py`
- `homeroom_defaults.py`

Timetable

Represents: Weekly lesson placement and timetable settings.

Ownership boundary: Timetable data depends on planning, teacher, subject, section, branch, and year context.

Must never be mixed:

1. Timetable edits must preserve scheduling constraints and scope.

Related files:

1. `routers/timetable.py`
1. `timetable_logic.py`

Academic Calendar

Represents: Events, academic dates, responsibilities, exports, and calendar settings.

Ownership boundary: Calendar records are branch/year/tenant scoped.

Must never be mixed:

1. Calendar events for one tenant or branch must not appear in another.

Related files:

1. `routers/academic_calendar.py`
1. `templates/academic_calendar.html`

Observations

Represents: Teacher observation records, feedback, scoring, evidence, and history.

Ownership boundary: Observation data is sensitive tenant operational data.

Must never be mixed:

1. Observation records must remain tenant-scoped and permission-protected.

Related files:

1. `routers/observations.py`
1. `templates/observation_*.html`

Knowledge Management System Artifacts

Represents: Markdown docs, ADRs, module history, generated PDF, manifest, and Knowledge Center read-only status.

Ownership boundary: Markdown under `docs/` is source of truth. Generated artifacts under `static/docs/` are snapshots.

Must never be mixed:

1. The app must not silently rewrite Markdown source docs.
1. The PDF must not be edited manually.
1. Protected documentation access should go through owner-only routes.

Related files:

1. `docs/`
1. `scripts/generate_docs_pdf.py`

- `static/docs/`
- `knowledge_service.py`

Tenant Isolation Rules

- Always know the current school group, branch, and academic year.
- Do not assume platform users have tenant scope.
- Do not assume SaaS accounts have operational records.
- Keep pending SaaS organization state separate from provisioned operational data.
- Preserve route permissions and service-level checks.
- Tests touching cross-tenant data should be treated as high value.

docs/engineering/DEVELOPMENT_STANDARDS.md

TIS Development Standards

These standards are mandatory for future human developers, Codex conversations, ChatGPT conversations, and technical reviewers working on TIS.

Inspect Before Editing

Before changing files:

1. inspect the current code with `rg` and targeted file reads,
1. understand the affected route/service/template/model,
1. read relevant docs, ADRs, and module history,
1. check tests for the touched module,
1. check the current branch and working tree.

Do not code from memory when local context is available.

Plan Before Implementation

For non-trivial work:

1. identify the affected modules,
1. identify tenant/permission/identity boundaries,
1. identify documentation updates,
1. identify tests or smoke checks,
1. keep the implementation path small.

Planning does not need to be long, but the risk boundaries must be clear.

Keep Changes Small And Scoped

1. Implement only the approved request.
1. Avoid unrelated refactors.
1. Avoid formatting churn in unrelated files.
1. Preserve existing patterns unless there is a clear reason to change them.
1. Do not move code across modules as a side effect.

Preserve Tenant Isolation

Tenant isolation is non-negotiable.

Always protect:

1. school group boundaries,
1. branch boundaries,
1. academic year scope,

- user role/permission scope,
- observation and teacher data sensitivity.

Any change that touches tenant scope should be treated as high risk.

Preserve Login And Platform Owner Flows

Do not casually change:

- `/login`,
- session cookies,
- platform owner detection,
- platform developer permissions,
- `/platform`,
- `/platform/knowledge`,
- scope switching,
- access denied handling.

Platform developers are not platform owners unless existing owner logic explicitly treats them that way.

Protect SaaS, Payment, And Provisioning

Do not casually change:

- `/saas/signup`,
- `/saas/login`,
- `/saas/account`,
- SaaS onboarding steps,
- Paddle checkout/client code,
- payment confirmation logic,
- billing state,
- provisioning readiness,
- provisioning jobs.

Webhook-confirmed payment is authoritative. Checkout return pages are not proof of payment.

Database And Migration Rules

- Never modify `tis.db` unless explicitly approved.
- Never modify migrations casually.
- Do not change `models.py` without understanding migration/repair implications.
- Do not add schema changes without tests and documentation.
- Never use destructive database operations without explicit approval.

Landing Website Rule

The public landing website is separate:

1. implementation: `tis-landing-website/`,
1. marketing docs: `docs/marketing/`,
1. ADRs: `0001`, `0007`.

Do not modify landing page code during backend or operational app tasks unless explicitly approved.

Protected Documentation Rule

1. Do not expose generated PDFs through direct public UI links.
1. Use owner-protected routes for PDF access.
1. Markdown source docs are reviewed development artifacts.
1. The app must not silently rewrite Markdown source docs.

Commit And Push Rule

1. Do not commit unless explicitly requested.
1. Do not push unless explicitly requested.
1. Do not merge unless explicitly requested.
1. Final reports should describe changes and validation clearly.

KMS Update Rule

Every meaningful implementation must update KMS if affected:

1. `docs/TIS_MASTER_CONTEXT.md`,
1. `docs/PROJECT_STATE.md`,
1. `docs/CHANGE_HISTORY.md`,
1. `docs/AI_PROJECT_CONTEXT.md`,
1. ADRs if major decisions changed,
1. module history for module-specific before/after state,
1. engineering docs if module maps, architecture, data model, standards, design philosophy, roadmap, or flows changed.

Regenerate the PDF if included docs changed.

Knowledge Impact Assessment

Every final implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
```

```
Reason if not updated:
```

A task is not complete until KIA is assessed.

Validation Standards

Run the narrowest meaningful validation:

1. compile checks for Python scripts/modules,
1. targeted pytest tests when behavior changes,
1. route/template smoke checks when pages are touched,
1. PDF generation when docs change,
1. frontend checks when landing/frontend code changes.

If validation cannot be run, say why.

docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

TIS UI/UX Design Philosophy

TIS should feel like a clean, premium academic SaaS platform. It should not feel like a generic admin dashboard, spreadsheet wrapper, or decorative marketing shell disconnected from real product value.

Overall Product Identity

Design principles:

1. professional,
1. light,
1. trustworthy,
1. academic,
1. calm,
1. data-aware,
1. premium without being flashy.

The product serves serious academic operations. UI should reduce confusion and support confident decisions.

Operational FastAPI App

The operational app should prioritize:

1. clarity,
1. role-based navigation,
1. data density,
1. fast scanning,
1. predictable controls,
1. strong permission boundaries,
1. branch/year context visibility.

Avoid:

1. generic admin clutter,
1. decorative cards that do not help workflows,
1. large marketing-style hero sections inside operational tools,
1. hiding important state behind vague labels,
1. layouts that make repeated operational use slow.

Operational pages should help users answer:

1. What am I looking at?
1. Which school/branch/year is active?
1. What needs action?

1. What can my role do here?
1. What changed after I saved?

Platform Owner Console

The Platform Owner Console should feel like a controlled operations console.

Priorities:

1. global visibility,
1. owner/developer separation,
1. clear organization switching,
1. safe access to sensitive tools,
1. explicit status labels,
1. no accidental tenant context confusion.

Avoid:

1. broad controls without owner-only checks,
1. exposing platform tools to developers by appearance alone,
1. unclear organization/branch state.

Knowledge Center

The Knowledge Center is an internal owner utility, not a marketing page.

Priorities:

1. KMS health,
1. source coverage,
1. freshness status,
1. protected PDF actions,
1. ADR/change/module visibility,
1. KIA policy reminder.

Avoid:

1. direct public static PDF links,
1. regenerate actions until approved,
1. app-side Markdown rewriting,
1. decorative presentation that hides status.

SaaS Onboarding Pages

SaaS onboarding should feel guided, calm, and customer-facing.

Priorities:

1. clear next step,

- progress visibility,
- plain customer language,
- reassurance around setup and billing,
- no internal engineering terms.

Customer-facing language should avoid:

- tenant,
- provisioning,
- M1/M2/M3/M4/M5,
- schema,
- migration,
- internal role mechanics.

Use customer language such as:

- organization,
- school,
- branch or campus,
- setup,
- plan,
- billing,
- account,
- getting your workspace ready.

Next.js Landing Website

The landing website should use premium storytelling and strong visual assets.

Priorities:

- clear problem/solution narrative,
- actual product credibility,
- school operations language,
- strong visual hierarchy,
- polished screenshots or generated assets when appropriate,
- conversion path into demo/signup.

Avoid:

- weak fake 3D visuals,
- generic SaaS gradients without product specificity,
- vague claims unsupported by product reality,
- internal terms like tenant/provisioning/milestones,

1 cluttered feature walls.

The landing page source of truth is `tis-landing-website/`, not legacy FastAPI landing files.

Visual System Direction

Future design system should support:

1 consistent cards, tables, filters, status badges, and action buttons,

1 clear empty/loading/error states,

1 accessible contrast,

1 stable layouts across modules,

1 consistent icon and label patterns,

1 role-aware navigation,

1 responsive behavior without losing operational density.

Tone And Copy

Internal app copy:

1 concise,

1 action-oriented,

1 operationally clear.

Customer-facing copy:

1 plain,

1 confident,

1 benefit-oriented,

1 free of internal implementation terms.

Platform owner copy:

1 precise,

1 status-driven,

1 explicit about access and risk.

docs/engineering/PRODUCT_ROADMAP.md

TIS Product Roadmap

This roadmap summarizes completed, current, next, and future product directions. It is not a release commitment; it is the current planning map for developers, owners, and reviewers.

Completed

SaaS Identity Foundation

Established SaaS account signup, login, account area, and the boundary between SaaS accounts, platform identities, and operational tenant users.

Pending Organizations Zone

Created pending organization structures and owner-facing review concepts for organizations moving through SaaS onboarding.

Plans And Billing Foundation

Added plan catalog, plan selection, billing status, checkout summary, checkout return/cancel, and payment/billing service boundaries.

Paddle Payment Collection

Added Paddle-oriented payment architecture with provider logic isolated behind service/client modules.

Tenant Provisioning Engine

Added provisioning workflows and jobs to create or connect operational school group/branch/year/user records from ready pending organizations.

KMS Foundation

Created source-of-truth Markdown docs, change history, ADRs, module history, AI project context, generated PDF booklet, and manifest.

Platform Owner Knowledge Center

Added protected owner-only Knowledge Center for KMS health, source freshness, manifest metadata, ADRs, module history, and protected PDF view/download.

KMS v3.0 Phase 3A

Added engineering handbook foundation with module map, repository architecture, user/system flows, and onboarding structure.

Current

KMS v3.0 Enhancement

Current KMS work expands the engineering handbook with database architecture, development standards, UI/UX philosophy, roadmap, and stronger developer/AI guidance.

Landing / Customer Experience Preparation

The product is preparing for stronger public customer journey work from landing page storytelling through SaaS signup, onboarding, billing, and operational access.

Next

Premium Landing Page Redesign

Improve public website quality, product storytelling, visuals, and conversion clarity while preserving the separate Next.js source-of-truth.

Customer Journey From Landing To Signup

Clarify the path from public website to SaaS signup, plan selection, onboarding, checkout, and workspace readiness.

SaaS Onboarding Refinement

Improve onboarding language, progress, validation, customer reassurance, and platform owner visibility.

Paddle Live Configuration

Configure live Paddle behavior when the payment account and production readiness are approved.

Subscription Lifecycle Management

Add clearer management for active subscriptions, renewal state, payment failures, and account lifecycle.

Feature Gating By Plan

Introduce plan-based access control for advanced capabilities, future AI features, and subscription tiers.

Customer Billing Portal

Provide customer-facing billing management if supported by payment architecture and approved product flow.

Trial / Upgrade / Downgrade / Cancellation Workflows

Define and implement subscription lifecycle actions safely with provider and local billing state alignment.

Future

AI Academic Assistant

Support academic leaders with intelligent recommendations based on verified tenant data.

AI Assessment Generation

Generate curriculum-aware assessments or question banks when curriculum/planning data is reliable enough.

AI Observation Improvement Plans

Assist supervisors with improvement plans, follow-up suggestions, and structured feedback from observation records.

AI Calendar Activity Planning

Suggest academic calendar activities, reminders, and coordination plans.

AI Dashboards And Academic Analytics

Provide richer academic insight, trends, risks, and multi-branch intelligence.

Multi-Branch Executive Dashboards

Give leadership cross-branch visibility into staffing, workload, observations, calendar, and planning health.

Curriculum And Planning Workflows

Expand from operational staffing/planning toward curriculum-aware academic planning if approved.

Reporting Improvements

Improve exports, dashboards, executive summaries, and decision-support reports.

Mobile Or Portal Expansion

Explore teacher/mobile/parent-style portals only if later approved and aligned with product strategy.

Roadmap Guardrails

1. Do not build AI features before data quality, permissions, and plan boundaries are clear.
1. Do not build subscription lifecycle actions before payment provider behavior is verified.
1. Do not redesign landing/customer journey without preserving the Next.js boundary.
1. Do not expose internal terms to customers.
1. Update KMS docs and roadmap after meaningful roadmap changes.

docs/adr/0001-separate-nextjs-landing-website.md

ADR 0001: Separate Next.js Landing Website

Status: Accepted

Date: 2026-06-26

Context

TIS needs a public marketing website and a protected operational application. The public site has different performance, design, routing, and deployment needs than the FastAPI operational portal.

Decision

Keep the public landing website in `tis-landing-website/` as a separate Next.js app. Keep the FastAPI app as the operational portal at `https://app.tisplatform.com`.

Alternatives Considered

1. Use FastAPI/Jinja for both marketing and app pages.
1. Put marketing pages inside the operational app templates.
1. Use a static site generated outside the repository.

Consequences

Positive:

1. Clear separation between marketing and operational concerns.
1. Landing page can evolve with a modern frontend stack.
1. Operational FastAPI app remains focused on authenticated workflows.

Tradeoffs:

1. Two runtimes must be understood.
1. Deployment and routing boundaries must be respected.

Related Docs / Files

1. `tis-landing-website/`
1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/TIS_MASTER_CONTEXT.md`
1. `templates/landing.html`
1. `static/landing/landing.css`

docs/adr/0002-separate-saas-identity-and-operational-users.md

ADR 0002: Separate SaaS Identity And Operational Users

Status: Accepted

Date: 2026-06-26

Context

TIS has SaaS account users who sign up, select plans, onboard organizations, and manage billing. It also has operational tenant users who work inside a provisioned school context. Platform owners and developers are separate platform identities.

Decision

Keep SaaS account identity, platform identity, and operational tenant identity distinct.

Alternatives Considered

1. Use one user table concept for all public signup, platform, and tenant users.
1. Automatically convert every SaaS signup into an operational tenant user.
1. Treat platform users as ordinary tenant administrators.

Consequences

Positive:

1. Clearer security boundaries.
1. Safer tenant isolation.
1. Billing/onboarding can proceed before operational provisioning.

Tradeoffs:

1. Identity flows require careful documentation.
1. Developers must avoid mixing account, platform, and tenant assumptions.

Related Docs / Files

1. `auth.py`
1. `models.py`
1. `saas/models.py`
1. `saas/service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0003-paddle-payment-architecture.md

ADR 0003: Paddle Payment Architecture

Status: Accepted

Date: 2026-06-26

Context

TIS requires subscription payments and billing visibility while preserving a clean boundary between academic operations and payment provider behavior.

Decision

Use Paddle behind service/client boundaries in the `saas/` package. Keep payment, pricing, billing, and currency logic separate from academic modules.

Alternatives Considered

1. Inline Paddle calls directly inside route handlers.
1. Store provider-specific behavior across operational modules.
1. Delay payment architecture until after provisioning.

Consequences

Positive:

1. Provider integration is isolated.
1. Academic workflows stay independent of payment details.
1. Billing status can be reasoned about separately from onboarding and provisioning.

Tradeoffs:

1. Service boundaries must be maintained.
1. Webhook/payment state must be carefully tested.

Related Docs / Files

1. `saas/paddle_client.py`
1. `saas/payment_service.py`
1. `saas/billing_service.py`
1. `saas/pricing_service.py`
1. `saas/currency_service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0004-webhook-only-payment-confirmation.md

ADR 0004: Webhook-Only Payment Confirmation

Status: Accepted

Date: 2026-06-26

Context

Checkout return pages can be reached by browser redirects, refreshes, or user navigation. They should not be treated as authoritative proof that payment succeeded.

Decision

Treat provider webhook confirmation as the authoritative source for successful payment state. Checkout return pages may inform the user but should not independently confirm payment or trigger provisioning as if payment is verified.

Alternatives Considered

1. Confirm payment based on checkout return route.
1. Allow manual UI confirmation from the SaaS account page.
1. Provision tenant records immediately after checkout redirect.

Consequences

Positive:

1. Reduces risk of false-positive payment confirmation.
1. Aligns billing state with provider-confirmed events.
1. Supports delayed provisioning after verified payment.

Tradeoffs:

1. Users may need clear pending-payment status while webhook processing completes.
1. Webhook handling must be reliable and observable.

Related Docs / Files

1. `saas/paddle_client.py`
1. `saas/payment_service.py`
1. `saas/billing_service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md

ADR 0005: Delayed Tenant Provisioning After Verified Payment

Status: Accepted

Date: 2026-06-26

Context

Public SaaS onboarding collects organization and branch information before an operational tenant exists. Provisioning creates operational school records and must be safe, reviewable, and recoverable.

Decision

Delay tenant provisioning until payment is verified and the organization is ready for provisioning. Use platform owner provisioning workflows and jobs to create or update operational tenant structures.

Alternatives Considered

1. Provision immediately after signup.
1. Provision immediately after checkout return.
1. Let public users create operational tenants directly.

Consequences

Positive:

1. Protects operational tenant data.
1. Keeps pending SaaS onboarding separate from live school records.
1. Allows platform owner oversight and retry behavior.

Tradeoffs:

1. More state transitions exist between signup and operational access.
1. Provisioning status must be communicated clearly.

Related Docs / Files

1. `saas/provisioning_service.py`
1. `saas/router.py`
1. `saas/service.py`
1. `templates/saas/admin_provisioning.html`
1. `templates/saas/admin_pending_organizations.html`
1. `docs/TIS_MASTER_CONTEXT.md`

docs/adr/0006-documentation-as-source-knowledge-management-system.md

ADR 0006: Documentation As Source / Knowledge Management System

Status: Accepted

Date: 2026-06-26

Context

TIS is growing across product, SaaS, architecture, payment, provisioning, landing, and operational modules. Future humans and AI coding assistants need reliable context and change history.

Decision

Use Markdown files under `docs/` as the source of truth. Generate the PDF booklet as a snapshot. Maintain change history, ADRs, module history, project state, master context, and AI project context as part of every meaningful development task.

Alternatives Considered

1. Keep knowledge only in chat history.
1. Keep only a generated PDF.
1. Let the app rewrite docs automatically.
1. Store decisions only in commits.

Consequences

Positive:

1. Project knowledge is reviewable and version-controlled.
1. Future Codex and ChatGPT conversations can onboard quickly.
1. Major decisions are preserved.

Tradeoffs:

1. Developers must maintain the KIA workflow.
1. Docs and PDF can become stale if the policy is ignored.

Related Docs / Files

1. `docs/README.md`
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`
1. `docs/CHANGE_HISTORY.md`
1. `docs/AI_PROJECT_CONTEXT.md`

```
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
```

docs/adr/0007-landing-page-visual-system-strategy.md

ADR 0007: Landing Page Visual System Strategy

Status: Accepted

Date: 2026-06-26

Context

The public landing page must communicate trust, product maturity, and academic operations value. It should not be accidentally changed during backend or operational app work.

Decision

Treat the Next.js landing website and its visual system as a separate product surface. Keep marketing content references under `docs/marketing/`. Do not modify landing design, copy, or legacy FastAPI landing files unless explicitly approved.

Alternatives Considered

1. Let backend tasks freely alter landing page presentation.
1. Continue using legacy FastAPI landing files as the public source.
1. Keep marketing strategy only in informal notes.

Consequences

Positive:

1. Protects brand and public conversion strategy.
1. Keeps landing work deliberate and reviewable.
1. Reduces accidental coupling with operational app changes.

Tradeoffs:

1. Landing page changes need explicit planning.
1. Developers must check the source-of-truth docs before editing public website files.

Related Docs / Files

1. `tis-landing-website/`
1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`
1. `docs/history/landing-page/README.md`

docs/history/academic-calendar/README.md

Academic Calendar History

This folder tracks meaningful changes to academic calendar workflows, event types, permissions, exports, and branch/year behavior.

Related files:

```
routers/academic_calendar.py
```

```
templates/academic_calendar.html
```

```
permission_registry.py
```

docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

2026-06-26 - KMS v3 Engineering Handbook

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Engineering Handbook

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only.

Previous Documented State

The KMS had master context, project state, ADRs, module history, AI context, generated PDF, manifest, and Platform Owner Knowledge Center docs. It did not yet contain a complete engineering handbook for module ownership, repository architecture, and end-to-end flows.

New Documented State

The KMS includes `docs/engineering/` with a module map, repository architecture guide, user/system flows, and developer onboarding index. The generated PDF now includes these docs as part of documentation version 3.0.

Reason For Change

New human developers and future Codex/ChatGPT conversations need a practical engineering handbook, not only a bundle of source documents.

User / Business Impact

Improves continuity, onboarding speed, technical review quality, and safer future implementation work.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page, database, migration, route, or runtime behavior changed.

Documentation Updated

- `docs/engineering/README.md`
- `docs/engineering/TIS_MODULE_MAP.md`
- `docs/engineering/REPOSITORY_ARCHITECTURE.md`
- `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
- `docs/TIS_MASTER_CONTEXT.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/README.md`
- `docs/CHANGE_HISTORY.md`

PDF Regenerated

Yes

Follow-Up Needed

Review PDF readability and handbook completeness before any Phase 3B work.

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md

2026-06-26 - KMS v3 Phase 3B Engineering Layers

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only.

Previous Documented State

The engineering handbook included the module map, repository architecture guide, user/system flows, and onboarding index. It did not yet include a database architecture overview, development standards, UI/UX philosophy, or product roadmap.

New Documented State

The engineering handbook includes Phase 3B layers:

1. database architecture overview,
1. development standards,
1. UI/UX design philosophy,
1. product roadmap,
1. stronger AI/human developer onboarding guidance.

Reason For Change

New senior developers, Codex conversations, ChatGPT conversations, and technical reviewers need stronger system boundaries, design expectations, roadmap context, and development rules before changing TIS.

User / Business Impact

Improves implementation safety, onboarding quality, technical review quality, and continuity across future development sessions.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`
1. `docs/engineering/DEVELOPMENT_STANDARDS.md`
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`

1 `docs/engineering/PRODUCT_ROADMAP.md`

1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review handbook completeness and readability before any Phase 3C work.

docs/history/engineering-handbook/README.md

Engineering Handbook History

This folder tracks meaningful changes to the TIS Engineering Handbook, including module maps, repository architecture, user/system flows, and onboarding guidance for humans and AI coding conversations.

Related files:

- `docs/engineering/README.md`
- `docs/engineering/TIS_MODULE_MAP.md`
- `docs/engineering/REPOSITORY_ARCHITECTURE.md`
- `docs/engineering/USER_AND_SYSTEM_FLOWS.md`

docs/history/landing-page/README.md

Landing Page History

This folder tracks meaningful changes to the public landing page, marketing positioning, visual system strategy, and source-of-truth boundaries.

Related docs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/marketing/landing_page_source_of_truth.md`
- `docs/marketing/tis_landing_page_master_content.md`

docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

2026-06-26 - Platform Owner Knowledge Center

Module: Platform Knowledge Center

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added Platform Owner Knowledge Center

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2C. Regenerate button, public access, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

Previous Documented State

TIS had KMS Markdown source files, ADRs, module history, a generated PDF booklet, and a manifest, but there was no protected in-app owner utility for viewing KMS status or accessing the booklet.

New Documented State

TIS has a read-only Platform Owner Knowledge Center that shows KMS health, manifest metadata, PDF freshness, source document status, recent change-history entries, ADRs, module history areas, and the Knowledge Impact Assessment checklist.

Reason For Change

Platform owners need a protected internal view of KMS status and a safe way to view/download the generated PDF without linking directly to static public paths.

User / Business Impact

Platform owners can verify whether the KMS snapshot is current and access the reference booklet from inside the app.

Technical Impact

Adds read-only KMS service helpers, owner-protected FastAPI routes, one app-shell template, and an owner-only Platform Console card. No SaaS, database, migration, billing, provisioning, or landing page behavior changes.

Documentation Updated

- `docs/TIS_MASTER_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/CHANGE_HISTORY.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/history/platform-knowledge/README.md`
- `docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md`

PDF Regenerated

Yes

Follow-Up Needed

Consider a future explicit owner-only regenerate action after review. The app must not silently rewrite Markdown source docs.

docs/history/platform-knowledge/README.md

Platform Knowledge Center History

This folder tracks meaningful changes to the owner-only TIS Knowledge Center, protected documentation PDF access, KMS metadata display, freshness detection, and knowledge health reporting.

Related files:

- `knowledge_service.py`
- `templates/platform_knowledge_center.html`
- `templates/platform_console.html`
- `main.py`

docs/history/provisioning/2026-06-26-kms-foundation.md

2026-06-26 - KMS Foundation

Module: Documentation and knowledge management

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Established Knowledge Management System Foundation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only.

Previous Documented State

TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal KMS policy, ADR structure, module history foundation, or AI onboarding context.

New Documented State

TIS now has a KMS foundation with policy, change history, ADRs, module history folders, and AI project context. The PDF generator includes these source docs and produces manifest metadata.

Reason For Change

Future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and reviewers need durable project context and history.

User / Business Impact

Improves continuity, reduces repeated rediscovery, and makes project decisions easier to review.

Technical Impact

No runtime app behavior changed. This is documentation and PDF generation foundation only.

Documentation Updated

1 `docs/CHANGE_HISTORY.md`

1 `docs/DOCUMENTATION_UPDATE_POLICY.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/adr/`

1 `docs/history/`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

PDF Regenerated

Yes

Follow-Up Needed

Phase 2C can later add a protected Platform Owner Knowledge Center after review.

docs/history/provisioning/README.md

Provisioning History

This folder tracks meaningful changes to pending organizations, provisioning jobs, verified-payment readiness, retry behavior, and platform owner provisioning oversight.

Related docs:

```
docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
```

```
docs/TIS_MASTER_CONTEXT.md
```

Related files:

```
saas/provisioning_service.py
```

```
saas/router.py
```

```
templates/saas/admin_provisioning.html
```

docs/history/README.md

TIS Module History

Module history stores deeper area-specific change records. It complements `docs/CHANGE_HISTORY.md`, which remains the chronological summary.

Use module history when a meaningful area changes and the previous documented state should be preserved.

Module Areas

```
    subscriptions/
    landing-page/
    academic-calendar/
    workforce-planning/
    saas-onboarding/
    provisioning/
```

Entry Template

```
# YYYY-MM-DD - Change Title

Module:
Related change-history entry:
Related ADRs:
Reviewer/approval notes:

## Previous Documented State

## New Documented State

## Reason For Change

## User / Business Impact

## Technical Impact

## Documentation Updated

## PDF Regenerated

## Follow-Up Needed
```

docs/history/saas-onboarding/README.md

SaaS Onboarding History

This folder tracks meaningful changes to signup, login, account, organization onboarding, contacts, branches, academic setup, review, and account self-service.

Related files:

- `saas/router.py`
- `saas/service.py`
- `templates/saas/`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

docs/history/subscriptions/README.md

Subscription History

This folder tracks meaningful changes to TIS subscription plans, pricing, billing status, payment behavior, checkout assumptions, and provider boundaries.

Related docs:

```
docs/adr/0003-paddle-payment-architecture.md
```

```
docs/adr/0004-webhook-only-payment-confirmation.md
```

```
docs/TIS_MASTER_CONTEXT.md
```

docs/history/workforce-planning/README.md

Workforce Planning History

This folder tracks meaningful changes to teacher information, workload planning, staffing gaps, planning sections, timetable relationships, and related reports.

Related files:

- `routers/teachers.py`
- `routers/planning.py`
- `routers/timetable.py`
- `teacher_capacity.py`
- `timetable_logic.py`

docs/marketing/landing_page_source_of_truth.md

Landing Page Source of Truth

The official source of truth for the public TIS landing website is:

`tis-landing-website/`

Public Website

- **Domain:** https://tisplatform.com
- **Runtime:** Next.js / Node
- **Local testing URL:** http://localhost:3000

All future marketing landing page changes must be made inside `tis-landing-website/`.

Application Portal

The TIS application portal remains separate from the public landing website:

- **Domain:** https://app.tisplatform.com
- **Runtime:** FastAPI / Python with PostgreSQL

Legacy Landing Page Files

The former FastAPI/Jinja landing page files are now legacy:

- `templates/landing.html`
- `static/landing/landing.css`

Codex and other developers must not modify these legacy files unless explicitly instructed.

docs/marketing/tis_landing_page_master_content.md

TIS Landing Page Master Content Approved Marketing Foundation

1 Main Positioning

TIS is a developing SaaS academic operations platform designed to connect school leadership, supervisors, teachers, branches, calendars, observations, staffing data, and decision-making inside one integrated system. The platform helps schools organize teacher information, analyze teaching loads, identify staffing gaps, coordinate shared academic calendars, manage structured observation cycles, support supervisor-teacher communication, and build clearer visibility across academic operations. As TIS continues to grow, advanced AI-powered features will be introduced through subscription-based plans, including curriculum-driven assessment generation, academic analytics, planning support, and intelligent recommendations based on verified school data.

1 Hero Section

Hero Badge Built for Schools Moving Beyond Spreadsheets Main Headline When Academic Decisions Are Scattered, Schools Lose Time, Clarity, and Control. Subheadline TIS brings teacher data, workloads, staffing gaps, academic calendars, observations, and leadership decisions into one connected SaaS platform - helping schools see what is happening, what is missing, and what needs action. CTA Buttons Request Early Access See How TIS Works Small Supporting Line A developing academic operations platform with advanced AI capabilities planned through subscription-based growth.

1 Problem Section

Section Label The Problem Main Heading Schools Are Not Short of Data. They Are Drowning in Disconnected Files. Section Text In many schools and educational organizations, every academic process becomes another spreadsheet. Teacher loads, subject coverage, observations, calendars, reports, staffing needs, and follow-up tasks are often managed in separate files, created by different people, in different formats. Over time, this creates a hidden operational burden. Staff members spend more time updating files than understanding the data. Leaders receive information late, inconsistently, or from multiple disconnected sources. In multi-branch organizations, the problem becomes even larger: every branch may work differently, every file may tell a different story, and every decision requires extra checking. Problem Cards

1 Too Many Spreadsheets

Academic work is scattered across different files, formats, and personal tracking systems.

1 Inconsistent Data

Each person or branch may structure information differently, making comparison and reporting difficult.

1 Heavy Staff Workload

Teachers, supervisors, and coordinators carry the burden of repeatedly entering and updating data.

1 Delayed Visibility

Leaders often discover staffing gaps, missing coverage, or academic issues after the problem has already grown.

1 Disconnected Decisions

Calendars, observations, staffing, reports, and academic planning are managed separately instead of working as one connected system.

1 Solution Section

Section Label The Solution Main Heading One Connected Platform to Bring Academic Operations Back Under Control. Section Text TIS brings the scattered pieces of school operations into one structured academic ecosystem. Instead of chasing separate spreadsheets, comparing different file formats, and waiting for updates from multiple sources, school leaders can work from a clearer, more connected view of what is happening across their school or organization. Teacher data, workloads, subject coverage, academic calendars, observations, supervision follow-up, branch structures, and leadership decisions become part of one system. This allows schools to reduce manual confusion, improve visibility, standardize academic processes, and make decisions with greater confidence. For multi-branch organizations, TIS helps create consistency across campuses while still allowing each branch to manage its own academic operations within a secure, organized, and tenant-isolated platform. Solution Cards

1 One Source of Academic Truth

Bring teacher data, workloads, observations, calendars, and staffing information into one connected system.

1 Clearer Leadership Visibility

Help principals, academic leaders, and supervisors see what is happening, what is missing, and what requires action.

1 Standardized Academic Processes

Reduce personal spreadsheet formats and create a more consistent way to manage academic operations across branches.

1 Smarter Staffing Decisions

Identify teaching load issues, uncovered hours, subject gaps, and staffing needs before they become larger problems.

1 Connected School Communication

Support better coordination between leadership, supervisors, teachers, and branches through shared academic workflows.

1 Ready for Future Intelligence

Build the structured data foundation needed for advanced AI-powered features as the platform continues to grow.

1 Core Platform Capabilities

Section Label Core Platform Capabilities Main Heading Everything Your Academic Operation Needs - Connected in One Platform. Section Intro TIS is designed to bring the most important academic operations into one structured SaaS environment. From teacher planning and branch management to observations, calendars, dashboards, and future AI-powered intelligence, the platform helps schools move from scattered work to connected academic control. Capability Cards

1 Academic Workforce Planning

Plan teacher assignments, analyze weekly loads, identify uncovered teaching hours, and understand staffing needs with greater clarity.

1 Academic Calendars & Shared Coordination

Coordinate academic events, timelines, school priorities, and branch-level planning through shared calendars that keep teams aligned.

1 Observation & Supervision Workflows

Manage structured teacher observations, supervisor feedback, shared records, and signing workflows inside one organized process.

1 Multi-Branch Organization Management

Support school groups, campuses, branches, users, roles, and tenant-isolated structures from one scalable SaaS platform.

1 Dashboards & Decision Visibility

Give leaders a clearer view of coverage, staffing gaps, teacher utilization, subject sufficiency, and academic follow-up needs.

1 Future AI-Powered Intelligence

Prepare for advanced subscription-based AI capabilities, including curriculum-driven assessment generation, answer keys, planning support, analytics, and intelligent recommendations based on verified academic data.

1 Teacher Workforce Planning Engine

Section Label Teacher Workforce Planning Engine Main Heading Know Exactly Who Is Teaching What - and Where Support Is Needed. Section Text One of the strongest current capabilities of TIS is its ability to help schools understand their teaching workforce with clarity. Instead of manually calculating teacher loads, subject coverage, and staffing needs across separate files, TIS brings this information into one structured planning engine. School leaders can track teacher qualifications, majors, subject specialties, weekly teaching loads, assigned subjects, uncovered hours, and additional staffing requirements. This helps academic teams identify gaps earlier, use existing teachers more effectively, and make staffing decisions based on clear data rather than assumptions. For schools and multi-branch organizations, this creates a more reliable way to plan academic coverage, compare staffing needs, and understand whether each subject, grade, and branch has the right teaching support. Key Capabilities

1 Teacher Load Visibility

See teacher workloads clearly and understand whether teaching hours are balanced, underused, or overloaded.

1 Subject Coverage Analysis

Identify which subjects are fully covered and which still have uncovered teaching hours.

1 Staffing Gap Detection

Detect shortages early before they affect schedules, instruction, or academic quality.

1 Qualification-Based Planning

Use teacher degrees, majors, and subject specialties to support smarter assignment decisions.

1 New Teacher Requirement Insights

Understand how many additional teachers may be needed based on uncovered hours and school workload requirements.

1 Multi-Branch Workforce Clarity

Support better staffing visibility across branches, campuses, and larger educational organizations.

1 Observation & Supervision Workflows

Section Label Observation & Supervision Workflows Main Heading Turn Teacher Observation into a Clear, Shared, and Accountable Process. Section Text TIS is designed to help schools move teacher observation away from scattered forms, disconnected files, and informal follow-up. Observation records, supervisor feedback, teacher responses, signatures, and approval steps can become part of one structured academic workflow. This gives supervisors a clearer way to document classroom visits, teachers a more transparent way to receive and respond to feedback, and school leaders a stronger view of supervision quality across departments and branches. As the platform continues to grow, advanced AI-assisted supervision features are planned to support supervisors by suggesting improvement plans based on observation notes, evaluation results, teacher performance evidence, and school-approved criteria. Key Capabilities

1 Structured Observation Records

Create organized observation forms and records that follow a clear school-approved process.

1 Shared Teacher-Supervisor Visibility

Allow teachers and supervisors to access relevant observation records, feedback, and follow-up actions.

1 Signing and Approval Workflow

Support formal documentation through review, acknowledgement, signing, and approval steps.

1 Supervision Follow-Up

Track feedback, recommendations, and improvement actions after classroom observations.

1 AI-Suggested Improvement Plans

Future AI-assisted tools may help supervisors generate suggested teacher improvement plans based on observation notes, evaluation data, and approved academic criteria.

1 Leadership Oversight

Give academic leaders better visibility over observation activity, supervision quality, and follow-up completion.

1 Reduced Observation Paperwork

Replace scattered observation files with a more consistent, traceable, and connected supervision system.

1 Academic Calendars & Shared Coordination

Section Label Academic Calendars & Shared Coordination Main Heading Turn the Academic Calendar into a Shared Planning Engine. Section Text TIS is designed to help schools move beyond static calendars and disconnected event planning. Academic leaders, supervisors, teachers, and branches can work from a shared academic calendar connected to the selected academic year, helping the whole school coordinate priorities, events, activities, observations, deadlines, and academic follow-up. Future calendar capabilities can support default worldwide and international academic events, allowing schools to plan meaningful activities around important global occasions. This gives international, national, private, and public schools a stronger way to connect school events with classroom learning, curriculum objectives, student engagement, and whole-school awareness. As TIS continues to grow,

AI-assisted calendar planning can help schools generate suggested activities for selected events, including classroom activities, outdoor activities, entertainment activities, curriculum-integrated tasks, and lesson-based ideas. The calendar vision also includes support for Sustainable Development Goals awareness planning, helping schools design year-round activities connected to SDGs, global citizenship, and curriculum integration. Key Capabilities

1 Academic-Year-Based Calendar

Build and manage school calendars according to the selected academic year, such as 2026-2027.

1 Shared School Coordination

Connect leaders, supervisors, teachers, and branches through one shared academic planning calendar.

1 International Event Awareness

Support worldwide and international academic events that schools can use for activities, campaigns, and curriculum connections.

1 AI-Suggested Activity Planning

Future AI-assisted tools may suggest activity plans based on selected events, school needs, grade levels, and curriculum links.

1 Curriculum-Integrated Events

Help schools connect calendar events with classroom learning, lesson planning, and academic objectives.

1 SDG Awareness Planning

Support year-round planning for Sustainable Development Goals awareness, student engagement, and global citizenship activities.

1 Branch-Level Visibility

Give school groups and multi-branch organizations a clearer view of academic events, priorities, and planning across campuses.

1 Multi-Branch Organization Management

Section Label Multi-Branch Organization Management Main Heading Built for Schools That Operate Across Branches, Campuses, and Teams. Section Text TIS is designed to support schools and educational organizations that need structure beyond a single location. Whether an organization manages one school, several branches, multiple campuses, or different academic pathways, TIS provides a scalable environment where users, roles, branches, and academic operations can be organized with greater clarity. School leadership and organization administrators can manage branches, users, permissions, branding, academic structures, and operational workflows inside a tenant-isolated SaaS environment. This helps every branch work within a shared organizational framework while still maintaining its own operational identity and academic needs. For educational groups, TIS can support stronger management-level visibility across branches. Leaders can monitor branch progress, compare completion levels, identify delayed tasks, and recognize where follow-up is needed across areas such as curriculum planning, exams, observations, calendars, reporting, and academic operations. Key Capabilities

1 Organization and Branch Structure

Manage organizations, schools, branches, campuses, and academic pathways inside one scalable platform.

1 Role-Based User Management

Assign users according to their responsibilities, such as organization administrator, branch leader, academic leader, supervisor, teacher, or future custom roles.

1 Tenant-Isolated Environment

Keep each organization's data, users, branding, and academic operations securely separated.

1 Branch Progress Intelligence

Give management-level users a clearer view of each branch's completion, progress, delays, and operational follow-up needs.

1 Branch Comparison Indicators

Compare branch performance across connected academic areas such as curriculum tasks, exams, observations, calendars, planning, and reports.

1 Custom Branding by Organization

Support organization and branch branding, including logos, visual identity, and school-specific presentation.

1 Scalable SaaS Structure

Prepare schools and educational groups for growth, future modules, subscription plans, and advanced AI-powered capabilities.

1 Dashboards & Decision Visibility

Section Label Dashboards & Decision Visibility Main Heading See Academic Progress, Risks, and Priorities Before They Become Problems. Section Text TIS dashboards are designed to give school leaders more than static reports. They provide a connected view of academic operations, helping leadership understand what has been completed, what is delayed, what needs follow-up, and where intervention may be required. As the platform continues to grow, dashboards can reflect progress across annual plans, weekly plans, curriculum coverage, assessment blueprints, quizzes, exams, academic calendar deadlines, teacher observations, supervision follow-up, staffing gaps, and branch-level performance. For branch management, dashboards can show the academic health of one school or campus. For organization-level leadership, dashboards can compare branches, highlight delayed tasks, identify performance gaps, and support faster, evidence-based decisions. Key Capabilities

1 Planning Completion Indicators

Track progress for annual plans, weekly plans, lesson planning, curriculum coverage, and completed academic tasks.

1 Assessment Progress Visibility

Monitor assessment blueprints, quizzes, exams, term-based assessment requirements, and calendar-linked due dates.

1 Observation & Supervision Insights

See observation progress, teacher follow-up status, supervisor actions, and areas requiring improvement plans.

1 Academic Calendar Integration

Connect events, deadlines, assessments, activities, and required tasks to dashboard indicators.

1 Teacher & Academic Performance Indicators

Support leadership with visual indicators related to teacher follow-up, academic progress, and school-level performance trends.

1 Branch and Organization Dashboards

Provide branch-level dashboards for school management and organization-level dashboards for comparing progress across multiple campuses.

1 Risk and Delay Alerts

Highlight delayed tasks, incomplete requirements, missing academic evidence, and branches or departments that need attention. Supporting Line: From reports to real-time academic visibility.

1 Future AI-Powered Intelligence

Section Label Future AI-Powered Intelligence Main Heading AI That Works Inside the Academic Workflow - Not Outside It. Section Text TIS is being developed with a future AI vision that goes beyond simple content generation. Instead of using AI as a separate tool, future AI-powered capabilities are planned to work inside the platform's academic workflows, supporting schools through the data already stored, reviewed, and approved in TIS. This future intelligence layer can support academic calendars, observation follow-up, curriculum planning, teacher support, assessment generation, dashboards, branch progress indicators, and leadership decision-making. At the center of this vision is a curriculum-to-classroom workflow. Uploaded curriculum data, standards, learning outcomes, annual plans, weekly plans, lesson plans, and confirmed instructional coverage can become the foundation for guided planning, AI-assisted recommendations, assessment generation, answer keys, and academic analytics. Future AI capabilities will be introduced progressively through subscription-based plans, allowing schools to access more advanced intelligence features as their needs grow. Key Capabilities

1 Curriculum-to-Classroom AI Support

Assist teachers and academic leaders in moving from uploaded curriculum to annual plans, weekly plans, lesson plans, and verified coverage.

1 AI-Assisted Assessment Generation

Support future generation of exams, quizzes, assessment blueprints, and answer keys based on approved curriculum data and confirmed instructional coverage.

1 AI-Supported Observation Follow-Up

Help supervisors generate suggested teacher improvement plans based on observation notes, evaluation results, and school-approved criteria.

1 AI-Powered Calendar Planning

Suggest activities for international events, SDG awareness, curriculum links, classroom activities, and school-wide academic events.

1 Academic Analytics and Recommendations

Provide future intelligent insights related to planning progress, teacher follow-up, assessment readiness, curriculum coverage, and branch performance.

1 Subscription-Based AI Growth

Introduce advanced AI features progressively through higher subscription plans, giving schools flexibility to grow into more powerful capabilities.

1 From Curriculum to Classroom - With Guided AI Support

Section Label From Curriculum to Classroom Main Heading Support Teachers Without Adding Another Burden. Section Text TIS is being developed to support teachers, not overload them. Instead of asking teachers to build annual plans, weekly plans, lesson plans, and curriculum tracking documents from scratch, future AI-assisted planning tools will help guide them through structured steps using approved templates, uploaded curriculum data, learning outcomes, standards, objectives, and school-specific requirements. Teachers will be able to review, adjust, complete, and submit their plans through the system. Academic leaders can then review and approve the work, turning planning documents into reliable, official academic data inside TIS. This helps reduce teacher workload, improve planning consistency, and give school leadership clearer visibility over what has been planned, approved, taught, and covered.

1 Subscription Plans Preview

Section Label Subscription Plans Main Heading Choose the Level of Academic Intelligence Your School Needs. Section Text TIS is being developed as a subscription-based SaaS platform, allowing schools and educational organizations to start with essential academic operations and grow into more advanced capabilities over time. As the platform expands, subscription plans can support different levels of access, from core teacher and academic operations to advanced dashboards, multi-branch management, supervision workflows, and future AI-powered intelligence. Plan Cards

1 Core

For schools that need a structured way to organize teacher data, academic operations, basic planning visibility, and essential workflows.

1 Professional

For schools that need stronger dashboards, shared academic calendars, observation workflows, staffing visibility, and improved coordination across teams.

1 Enterprise AI

For school groups and larger organizations that need multi-branch visibility, advanced customization, branch progress intelligence, AI-assisted planning, assessment generation, analytics, and strategic decision support. Supporting Note AI-powered capabilities will be introduced progressively and may vary according to subscription level, school needs, and platform development stage. CTA Request Early Access Request Pricing

1 Final Call-to-Action

Section Label Start Your TIS Journey Main Heading Ready to Bring Clarity, Control, and Intelligence to Your Academic Operations? Section Text TIS is being built for schools and educational organizations that want to move beyond scattered files, disconnected workflows, and delayed decisions. Start exploring how one connected academic operations platform can support your leadership team, supervisors, teachers, branches, and future AI-powered growth. CTA Buttons Request Early Access Book a Demo Supporting Line TIS is currently in active development, with selected features and advanced AI capabilities planned for progressive release through subscription-based plans.

docs/location-data-roadmap.md

TIS Location Data Roadmap

Phase 1: Global Form Usability

Status: implemented in the current working tree.

Phase 1 intentionally keeps location data attached directly to an organization or branch. It does not create shared locality records.

Included:

1. Controlled country selection from the local dataset.
1. Controlled region/state/province selection when dataset coverage exists.
1. Dataset-backed city/locality selection.
1. `Other / manual entry` fallback for missing regions and cities.
1. Optional district and neighborhood fields, stored separately from city.
1. Local cached APIs; no external location API dependency at runtime.
1. Existing text location fields remain compatible with legacy Saudi data.

Manual region and city values are record-level values. They are not published to other tenants and are not added to the imported dataset.

Deferred Architecture

The following phases are roadmap items only. They are not part of Phase 1 and must not be inferred from the current text-based storage model.

Locality Registry

Create stable TIS locality identifiers and canonical locality records that are independent from vendor dataset identifiers. Organizations and branches would eventually reference these records while retaining display-name snapshots for audit and migration compatibility.

Locality Aliases

Support native names, translated names, transliterations, spelling variants, and historical names. Alias matching should be region-aware and must not merge places solely because their normalized names match.

GeoNames and Official Dataset Enrichment

Build an offline enrichment process using GeoNames country dumps and, where available, official national gazetteers. Source priority, licensing, attribution, place classification, and duplicate resolution must be defined before combining records.

Tenant-Private City Additions

Allow an authorized tenant user to suggest a missing locality. New records should initially be visible only to that tenant and must be constrained to the tenant's organization scope.

Platform Moderation

Provide a Platform Owner moderation queue for reviewing proposed localities, potential duplicates, coordinates, place type, aliases, and source evidence.

City Verification and Promotion

Support explicit states such as tenant-private, pending, verified, deprecated, and merged. Promotion to the global catalog must be an audited platform action, never an automatic consequence of a tenant entering free text.

Dataset Refresh and ETL Workflow

Introduce versioned, offline imports with source checksums, staging tables, coverage metrics, diff review, attribution records, and rollback support. Referenced localities should be deprecated or merged rather than deleted.

Guardrails for Future Work

1. Preserve tenant isolation for all tenant-created locality data.
1. Keep stable internal IDs separate from external source IDs.
1. Retain source and license provenance for every imported record.
1. Do not call public geocoding services from normal page requests.
1. Do not modify the vendor JSON to store tenant-entered values.
1. Migrate existing organization and branch text values without data loss.